

রিকারেন্ট নিউরাল নেটওয়ার্ক (RNN)

থেকে LSTM পর্যন্ত: সম্পূর্ণ স্টাডি নোট

*Theory, Mathematical Foundations, Hand-Worked Forward & Backward Calculation,
and the Role of LSTM*

*Sequential Data → Need for Memory → RNN → Forward Pass → BPTT → Vanishing / Exploding
Gradient → LSTM*

১. Sequential Data – সিকোয়েন্সিয়াল ডাটা

১.১ সিকোয়েন্সিয়াল ডাটা (Sequential Data) কী?

Sequential Data বলতে এমন এক ধরনের ডাটা বোঝানো হয়, যেখানে প্রতিটি উপাদানের (element) মান শুধুমাত্র তার নিজস্ব বৈশিষ্ট্যের উপর নির্ভর করে না – বরং এটি কোন ক্রমে (order) এবং কোন প্রেক্ষাপটে (context) এসেছে তার উপরেও নির্ভর করে। অর্থাৎ ডাটার মধ্যে একটি Temporal Dependency অথবা Positional Dependency বিদ্যমান থাকে – ক্রম পরিবর্তন করলে অর্থ বা তথ্য বদলে যায়।

সংক্ষেপে: যদি কোনো ডাটাসেটের একটি স্যাম্পলের মান বোঝার জন্য তার আগের বা পরের স্যাম্পলগুলো জানা প্রয়োজন হয়, তবে সেটি Sequential Data।

১.২ বাস্তব-জীবনের উদাহরণ (Real-World Examples)

ডোমেইন (Domain)	উদাহরণ (Example)	কেন ক্রম গুরুত্বপূর্ণ
Text (ভাষা)	“আমি ভাত খাই” বনাম “ভাত আমি খাই”	শব্দের ক্রম বদলালে বাক্যের ব্যাকরণ ও স্বাভাবিকতা বদলে যায়
Time-Series	শেয়ার-বাজারের প্রতিদিনের মূল্য, তাপমাত্রা রিডিং	আজকের মান গতকালের মানের উপর নির্ভরশীল (autocorrelation)
Audio / Speech	কণ্ঠস্বরের waveform, phoneme sequence	তরঙ্গের ক্রম বদলালে সম্পূর্ণ ভিন্ন উচ্চারণ তৈরি হয়
DNA / Genomics	নিউক্লিওটাইড সিকোয়েন্স (A, T, G, C)	সিকোয়েন্সের ক্রম প্রোটিনের গঠন নির্ধারণ করে

ডোমেইন (Domain)

উদাহরণ (Example)

কেন ক্রম গুরুত্বপূর্ণ

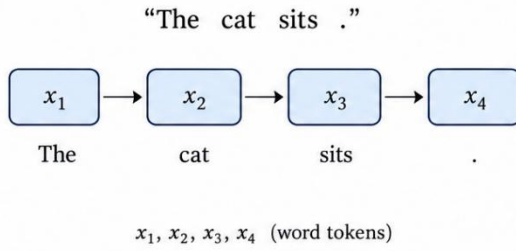
চিত্র / Figure 1: Sequential Data-এর বিভিন্ন রূপ

এই ডায়াগ্রামে যা দেখানো হবে (What this diagram should show):

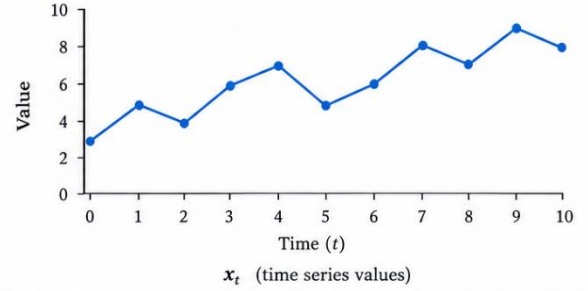
- চারটি প্যানেলে চারটি ডোমেইন দেখানো: Text, Time-Series, Audio, DNA
- Text: শব্দের একটি সারি $x_1, x_2, x_3, \dots$ বাক্স আকারে, তীরচিহ্ন দিয়ে ক্রম বোঝানো
- Time-Series: একটি সাধারণ লাইন-গ্রাফ, x-অক্ষে সময়, y-অক্ষে মান
- Audio: একটি waveform সময়ের সাপেক্ষে
- DNA: A-T-G-C অক্ষরের একটি সারি, নিউক্লিওটাইড আকারে রঙিন ব্লকে

Four Types of Sequential Data

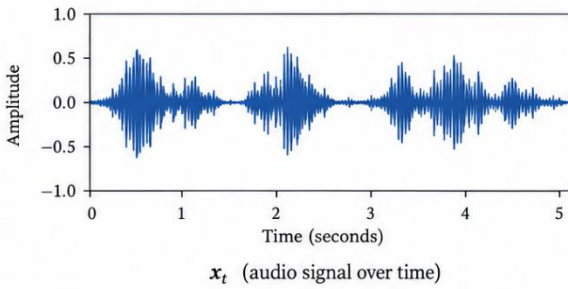
1. Text Sequence (Sentence)



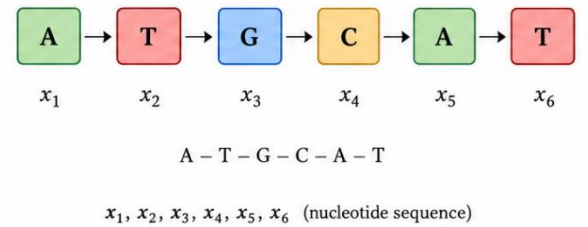
2. Time-Series Data



3. Audio Sequence



4. DNA Sequence



১.৩ কেন ক্রম গুরুত্বপূর্ণ (Why Order Matters)

Sequential Data-এ শুধু “কী আছে” তা নয়, “কোন অবস্থানে আছে” তা-ও গুরুত্বপূর্ণ তথ্য বহন করে। উদাহরণস্বরূপ, “কুকুর বিড়ালকে তাড়া করল” এবং “বিড়াল কুকুরকে তাড়া করল” — দুটি বাক্য ঠিক একই শব্দ ব্যবহৃত হয়েছে, কিন্তু ক্রম ভিন্ন হওয়ায় অর্থ সম্পূর্ণ বিপরীত। তাই যেকোনো মডেলকে এই ধরনের ডাটা বুঝতে হলে position ও context উভয়ই ধরে রাখতে হবে।

Key Takeaways

- Sequential Data-এ প্রতিটি উপাদানের অর্থ তার ক্রম ও প্রেক্ষাপটের উপর নির্ভরশীল।
- Text, Time-Series, Audio, DNA — সবই Sequential Data-এর বাস্তব উদাহরণ।
- ক্রম পরিবর্তন করলে ডাটার প্রকৃত অর্থ বদলে যেতে পারে (positional dependency)।
- এই বৈশিষ্ট্যটিই পরবর্তীতে দেখাবে কেন সাধারণ ANN এই ধরনের ডাটার জন্য উপযুক্ত নয়।

২. Standard ANN কেন Sequence ডাটায় ব্যর্থ হয়?

একটি সাধারণ Feed-forward Artificial Neural Network (ANN) ধরে নেয় যে ইনপুট ফিচারগুলো একে অপরের থেকে স্বাধীন (i.i.d. — independent and identically distributed) এবং তাদের মধ্যে কোনো টেম্পোরাল সম্পর্ক নেই। এই অনুমানের কারণে Sequential ডাটাতে ANN-এর তিনটি মৌলিক দুর্বলতা দেখা যায়।

২.১ Lack of Memory — কোনো স্মৃতি বা Recurrence নেই

ANN-এর প্রতিটি forward pass সম্পূর্ণভাবে স্বতন্ত্র (stateless)। একটি ইনপুট প্রসেস করার সময় নেটওয়ার্কের কাছে পূর্ববর্তী ইনপুট সম্পর্কে কোনো তথ্য সংরক্ষিত থাকে না। ফলে x_t প্রসেস করার সময় x_{t-1} সম্পর্কে নেটওয়ার্কের কোনো ধারণা (context) থাকে না — যা ভাষা বা টাইম-সিরিজ বোঝার জন্য অপরিহার্য।

২.২ Sequence Order ধারণ করতে অক্ষমতা

ANN-এর ইনপুট লেয়ারে সমস্ত ফিচার একসাথে, সমান্তরালভাবে (parallel) প্রবেশ করে — যেমন Bag-of-Words পদ্ধতিতে একটি বাক্যের সব শব্দ গণনা করা হয়, কিন্তু কোন শব্দ আগে বা পরে এসেছে তা সংরক্ষিত হয় না। ফলে “কুকুর বিড়ালকে তাড়া করল” এবং “বিড়াল কুকুরকে তাড়া করল” — সম্পূর্ণ ভিন্ন অর্থের এই দুটি বাক্য ANN-এর কাছে একই রকম ইনপুট মনে হতে পারে।

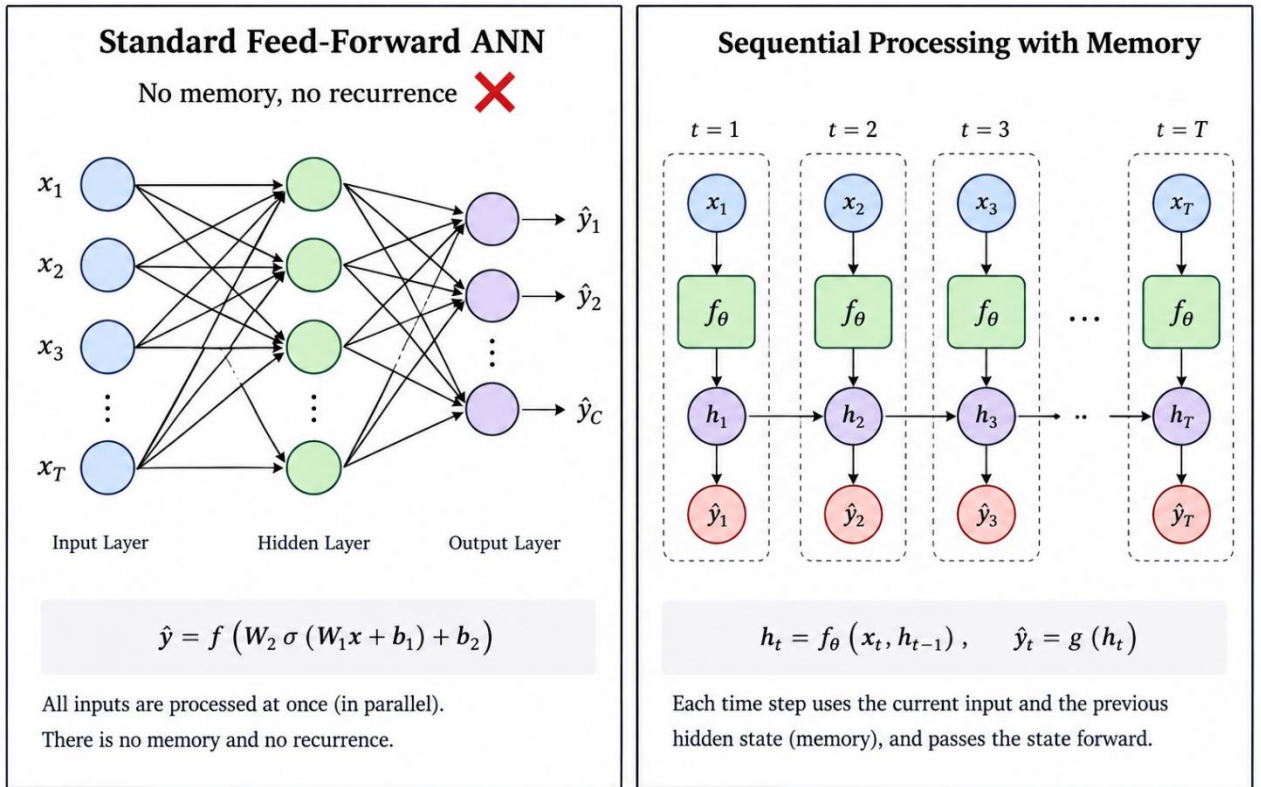
২.৩ Fixed-Length Input Constraint

ANN-এর ইনপুট লেয়ারের নিউরন সংখ্যা architecturally পূর্বনির্ধারিত ও স্থির থাকে। কিন্তু বাস্তব-জগতের সিকোয়েন্স — যেমন বাক্য, অডিও ক্লিপ, বা DNA স্ট্র্যান্ড — এর দৈর্ঘ্য পরিবর্তনশীল (variable-length)। একটি স্থির-আকারের ইনপুট লেয়ার দিয়ে পরিবর্তনশীল-দৈর্ঘ্যের ডাটা স্বাভাবিকভাবে হ্যান্ডেল করা সম্ভব নয়।

চিত্র / Figure 2: ANN বনাম Sequential Data

এই ডায়াগ্রামে যা দেখানো হবে (What this diagram should show):

- বামে: একটি সাধারণ feed-forward ANN, যেখানে সব ইনপুট নিউরন একসাথে সমান্তরালে প্রবেশ করছে, কোনো ফিডব্যাক লুপ নেই
- ডানে: একই ইনপুটকে সময়ের ক্রমে (sequential) দেখানো, প্রতিটি ধাপে একটি ছোট মেমোরি/হিডেন-স্টেট আইকন যুক্ত
- বাম প্যানেলে একটি লাল × চিহ্ন, বোঝাতে যে সময়ের তথ্য হারিয়ে যাচ্ছে
- নিচে ছোট ক্যাপশন: “ANN has no memory across inputs”



Common Mistakes

- মনে করা যে ANN বড় বা বেশি লেয়ার ব্যবহার করলেই sequence বুঝতে পারবে — সমস্যাটি architecture-এর, গভীরতার নয়।
- Bag-of-Words-কে sequence-aware representation ভেবে ভুল করা — এটি ক্রম সংরক্ষণ করে না।
- Fixed-length input constraint-কে শুধু “মেমোরি সমস্যা” হিসেবে ভাবা, যদিও এটি একটি পৃথক architectural সীমাবদ্ধতা।

৩. RNN-এর মূল ধারণা (Core Idea of RNN)

এই তিনটি সমস্যার সমাধান হিসেবে Recurrent Neural Network (RNN) একটি নতুন ধারণা প্রবর্তন করে: নেটওয়ার্কের ভেতর একটি “মেমোরি” বা “Hidden State” যুক্ত করা, যা প্রতিটি time-step-এ নিজেকে আপডেট করে এবং পূর্ববর্তী তথ্যের একটি সংক্ষিপ্ত রূপ (context) ধরে রাখে। সংক্ষেপে বলা যায়:

$$\text{RNN} = \text{ANN} + \text{Memory / Context}$$

৩.১ Hidden State

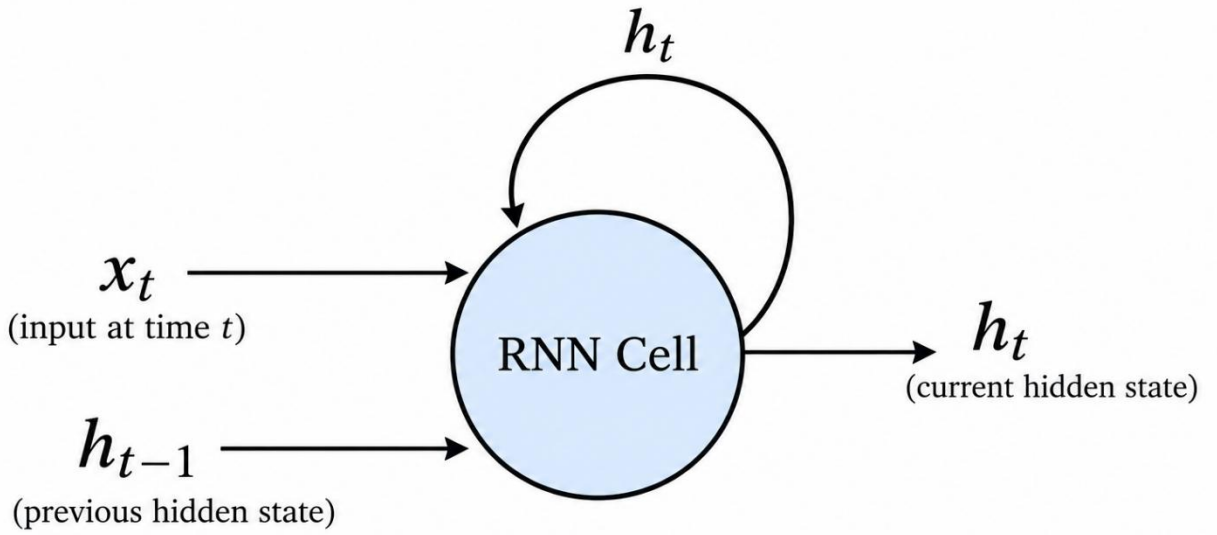
Hidden state (h_t) হলো RNN-এর “মেমোরি” — এটি একটি ভেক্টর যা এখন পর্যন্ত দেখা সমস্ত ইনপুটের একটি সংক্ষিপ্ত, ঘনীভূত (compressed) সারমর্ম ধারণ করে। প্রতিটি time-step-এ hidden state আগের hidden state (h_{t-1}) এবং বর্তমান ইনপুট (x_t) থেকে নতুন করে গণনা করা হয়।

৩.২ Recurrence (পুনরাবৃত্তি)

Recurrence মানে হলো, একই ফাংশন বারবার প্রয়োগ করা — প্রতিটি time-step-এ hidden state তার নিজের আগের মানের উপর নির্ভর করে আপডেট হয়। এই feedback loop-টিই RNN-কে “রিকারেন্ট” (recurrent) নাম দিয়েছে:

চিত্র / Figure 3: RNN-এর মূল ধারণা (Core Idea)**এই ডায়াগ্রামে যা দেখানো হবে (What this diagram should show):**

- একটি একক ব্লক "RNN Cell" আঁকা, যার ইনপুট হিসেবে x_t এবং h_{t-1} প্রবেশ করছে
- আউটপুট হিসেবে h_t বের হচ্ছে
- h_t থেকে একটি বাঁকানো তীর (curved arrow) ফিরে গিয়ে পরবর্তী time-step-এর ইনপুটে প্রবেশ করছে — feedback loop
- সাথে ছোট ক্যাপশন: "Same cell reused at every time step"

**৩.৩ Parameter Sharing**

RNN দর্শনের মূল ভিত্তি হলো Parameter Sharing এবং Recurrence — একই ওয়েট ম্যাট্রিক্স প্রতিটি time-step-এ পুনরায় ব্যবহৃত হয়, এবং প্রতিটি স্টেপের আউটপুট (hidden state) পরবর্তী স্টেপের ইনপুটের একটি অংশ হিসেবে ফিরে আসে (feedback loop)। ফলে নেটওয়ার্কটি “সময়ের সাথে সাথে” (through time) তথ্য বহন করতে সক্ষম হয়, এবং তাত্ত্বিকভাবে যেকোনো দৈর্ঘ্যের সিকোয়েন্স প্রসেস করতে পারে — কারণ প্যারামিটার সংখ্যা সিকোয়েন্স দৈর্ঘ্যের উপর নির্ভর করে না।

Key Takeaways

- Hidden state (h_t) হলো RNN-এর মেমোরি — এটি এখন পর্যন্ত দেখা তথ্যের সংক্ষিপ্ত রূপ ধারণ করে।
- Recurrence-এর মাধ্যমে h_{t-1} থেকে h_t গণনা করা হয় — একটি feedback loop তৈরি হয়।
- Parameter Sharing-এর কারণে RNN যেকোনো দৈর্ঘ্যের সিকোয়েন্সে কাজ করতে পারে, প্যারামিটার সংখ্যা বৃদ্ধি না করেই।
- RNN = ANN + Memory — এই একটি লাইনেই RNN-এর মূল দর্শন ধরা পড়ে।

8. Sequential Data প্রস্তুতকরণ (Preparing Sequential Data)

8.1 Variable-Length Sequence সমস্যা

বাস্তব সিকোয়েন্সগুলোর দৈর্ঘ্য ভিন্ন ভিন্ন হয় — একটি বাক্যে ৫টি শব্দ, আরেকটিতে ১২টি। কিন্তু নিউরাল নেটওয়ার্ককে ব্যাচ (batch) আকারে প্রসেস করার জন্য একই আকারের টেনসর প্রয়োজন হয়, যাতে GPU-তে সমান্তরাল (vectorized) গণনা করা যায়। এই দ্বন্দ্ব সমাধানের জন্য Zero-Padding কৌশল ব্যবহার করা হয়।

8.2 Zero Padding

ধারণা: একটি ব্যাচের মধ্যে সবচেয়ে দীর্ঘ সিকোয়েন্সের দৈর্ঘ্য ($T_{\max}$) নির্ধারণ করা হয়, এবং ছোট সিকোয়েন্সগুলোর শেষে (বা শুরুতে) শূন্য (0) যোগ করে সবাইকে সমান দৈর্ঘ্যে আনা হয়।

উদাহরণ (Post-padding, $T_{\max} = 5$):

Sequence A: [3, 7, 2] → [3, 7, 2, 0, 0]

Sequence B: [5, 1, 8, 4, 6] → [5, 1, 8, 4, 6] (already length 5)

সুবিধা: সমগ্র ব্যাচকে একটি নিয়মিত (rectangular) টেনসরে রূপান্তর করা যায়, যা vectorized প্রসেসিং সম্ভব করে।

অসুবিধা: (ক) Computational Overhead — মডেলকে অপ্রয়োজনীয় শূন্য টোকেনের উপরেও গণনা চালাতে হয়; (খ) Memory Overhead — একটি ব্যাচে দৈর্ঘ্যের পার্থক্য বেশি হলে অতিরিক্ত প্যাডিং প্রচুর মেমোরি অপচয় করে।

8.3 Masking

এই সমস্যা কমাতে Masking ব্যবহার করা হয়, যাতে loss গণনার সময় প্যাডেড অংশগুলো উপেক্ষা করা যায়।

Observation

- প্যাডিং ভ্যালু (সাধারণত ০) কখনোই প্রকৃত ইনপুট হিসেবে গণ্য করা উচিত নয় — এটি শুধুমাত্র ব্যাচের আকার সমান করার জন্য একটি প্লেসহোল্ডার। Masking ছাড়া মডেল ভুলভাবে প্যাডিং-কে অর্থপূর্ণ ডাটা হিসেবে শিখে ফেলতে পারে।

8.8 Tokenization

টেক্সট ডাটাকে সংখ্যায় রূপান্তরের প্রথম ধাপ হলো Tokenization — বাক্যকে ক্ষুদ্র একক (শব্দ বা সাব-ওয়ার্ড টোকেন)-এ ভাগ করা।

উদাহরণ — বাক্য: “আমি ভাত খাই”

Tokenization → [আমি, ভাত, খাই]

8.৫ Vocabulary

সম্পূর্ণ কর্পাসের সব ইউনিক শব্দ সংগ্রহ করে একটি অভিধান V তৈরি করা হয়, যেখানে প্রতিটি শব্দকে একটি ইউনিক ইনডেক্স দেওয়া হয়। $|V|$ = অভিধানের আকার (vocabulary size)।

$V = \{ \text{আমি, ভাত, খাই, স্কুলে, যাই} \}, |V| = 5$

8.৬ One-Hot Encoding

প্রতিটি শব্দকে $|V|$ -মাত্রিক একটি ভেক্টরে রূপান্তর করা হয়, যেখানে শুধুমাত্র সেই শব্দের ইনডেক্স-পজিশনে মান 1 এবং বাকি সব পজিশনে মান 0 থাকে।

শব্দ (Word)	ইনডেক্স (Index)	One-Hot Vector
আমি	0	[1, 0, 0, 0, 0]
ভাত	1	[0, 1, 0, 0, 0]
খাই	2	[0, 0, 1, 0, 0]
স্কুলে	3	[0, 0, 0, 1, 0]
যাই	4	[0, 0, 0, 0, 1]

8.৭ One-Hot Encoding-এর সীমাবদ্ধতা (Limitations)

High Dimensionality / Sparsity

বাস্তব-জীবনের অভিধানে সাধারণত লক্ষাধিক (১০⁵ থেকে ১০⁶) ইউনিক শব্দ থাকে। ফলে প্রতিটি শব্দের ভেক্টর অত্যন্ত উচ্চ-মাত্রিক (high-dimensional) কিন্তু প্রায় সম্পূর্ণ শূন্য (sparse) পূর্ণ হয় — মেমোরি ও কম্পিউটেশনের দিক থেকে অত্যন্ত অদক্ষ।

অর্থগত সম্পর্কের অভাব (Lack of Semantic Relationship)

One-hot representation-এ প্রতিটি ভেক্টর একে অপরের থেকে সমদূরত্বে (orthogonal) থাকে — যেকোনো দুটি ভিন্ন শব্দের ভেক্টরের dot product সর্বদা শূন্য হয়। এর মানে, মডেলের দৃষ্টিতে “রাজা” এবং “রানী” শব্দ দুটি ঠিক ততটাই “ভিন্ন” যতটা “রাজা” এবং “কলা” — যদিও বাস্তবে প্রথম জোড়ার মধ্যে সুস্পষ্ট অর্থগত সম্পর্ক আছে। One-hot vector এই সম্পর্ক ধারণ করতে সম্পূর্ণ অক্ষম।

8.৮ Word Embeddings

এই সীমাবদ্ধতা কাটিয়ে ওঠার জন্য Word Embeddings (যেমন Word2Vec, GloVe) ব্যবহার করা হয়, যেখানে প্রতিটি শব্দকে একটি তুলনামূলক নিম্ন-মাত্রিক (৫০–৩০০ মাত্রা), ঘন (dense) এবং প্রশিক্ষণযোগ্য (trainable) ভেক্টরে ম্যাপ করা হয়। এই ভেক্টর-স্পেসে অর্থগতভাবে সম্পর্কিত শব্দগুলো কাছাকাছি অবস্থান করে — যেমন বিখ্যাত সম্পর্ক: $\text{vector}(\text{“রাজা”}) - \text{vector}(\text{“পুরুষ”}) + \text{vector}(\text{“নারী”}) \approx \text{vector}(\text{“রানী”})$ । এটি RNN-ভিত্তিক NLP মডেলের প্রথম লেয়ার (Embedding Layer) হিসেবে ব্যাপকভাবে ব্যবহৃত হয়।

বৈশিষ্ট্য	One-Hot Encoding	Word Embedding
মাত্রা (Dimension)	উচ্চ ($ V $, প্রায়ই 10^5+)	নিম্ন (সাধারণত ৫০–৩০০)
Density	Sparse (প্রায় সবই শূন্য)	Dense (ঘন)
Trainable?	না	হ্যাঁ
Semantic সম্পর্ক	ধারণ করে না	আংশিকভাবে ধারণ করতে পারে
Observation Word Embedding “সবসময়” নিখুঁতভাবে semantic সম্পর্ক এনকোড করে — এমন দাবি সঠিক নয়। এটি শুধু ডাটা ও training objective-এর উপর ভিত্তি করে আংশিক ও পরিসংখ্যানগত সম্পর্ক শেখে, নিখুঁত অর্থগত বোঝাপড়া নয়।		

বৈশিষ্ট্য	One-Hot Encoding	Word Embedding
Common Mistakes • প্যাডিং ভ্যালুকে (০) প্রকৃত ইনপুট ডাটা হিসেবে গণনা করা – masking ছাড়া এটি ভুল ফলাফল দিতে পারে। • One-hot vector-কে “সস্তা এবং যথেষ্ট” ভাবা বড় vocabulary-র ক্ষেত্রে – memory ও compute উভয় দিক থেকেই এটি অদক্ষ। • Word Embedding-কে semantic understanding-এর নিখুঁত সমাধান ভাবা – এটি কেবল একটি উপযোগী approximation।		
Key Takeaways • Zero-Padding ব্যাচের সব সিকোয়েন্সকে সমান দৈর্ঘ্যে আনে, কিন্তু Masking ছাড়া এটি ভুল ফলাফল দিতে পারে। • Tokenization → Vocabulary → One-Hot হলো টেক্সটকে সংখ্যায় রূপান্তরের ক্রমিক ধাপ। • One-Hot Encoding উচ্চ-মাত্রিক, sparse, এবং semantic সম্পর্কহীন। • Word Embedding নিম্ন-মাত্রিক, dense, trainable, এবং আংশিক semantic সম্পর্ক ধারণ করতে পারে।		

৫. RNN আর্কিটেকচার (Architectures)

ইনপুট এবং আউটপুট সিকোয়েন্সের দৈর্ঘ্যের উপর ভিত্তি করে RNN-কে চারটি মূল শ্রেণিতে ভাগ করা যায়। এই শ্রেণিবিভাগ বোঝা গুরুত্বপূর্ণ কারণ বাস্তব সমস্যার ধরন অনুযায়ী সঠিক আর্কিটেকচার বেছে নিতে হয়।

ধরন (Type)	ইনপুট → আউটপুট	ব্যবহারিক উদাহরণ (Use Case)
One-to-One	একক → একক	প্রচলিত Feed-forward ANN / Image Classification
One-to-Many	একক → সিকোয়েন্স	Image Captioning
Many-to-One	সিকোয়েন্স → একক	Sentiment Analysis
Many-to-Many	সিকোয়েন্স → সিকোয়েন্স	Machine Translation, Video Frame Classification

Observation

- One-to-One আসলে কঠোরভাবে recurrent architecture নয় — এটি শুধু baseline হিসেবে দেখানো হয়, কারণ এখানে কোনো time-step-ব্যাপী recurrence নেই।

৫.১ One-to-One

একটি একক ইনপুট থেকে একটি একক আউটপুট তৈরি হয় — এটি মূলত একটি প্রচলিত feed-forward নেটওয়ার্কের সমতুল্য, কোনো recurrence ছাড়াই।

৫.২ One-to-Many

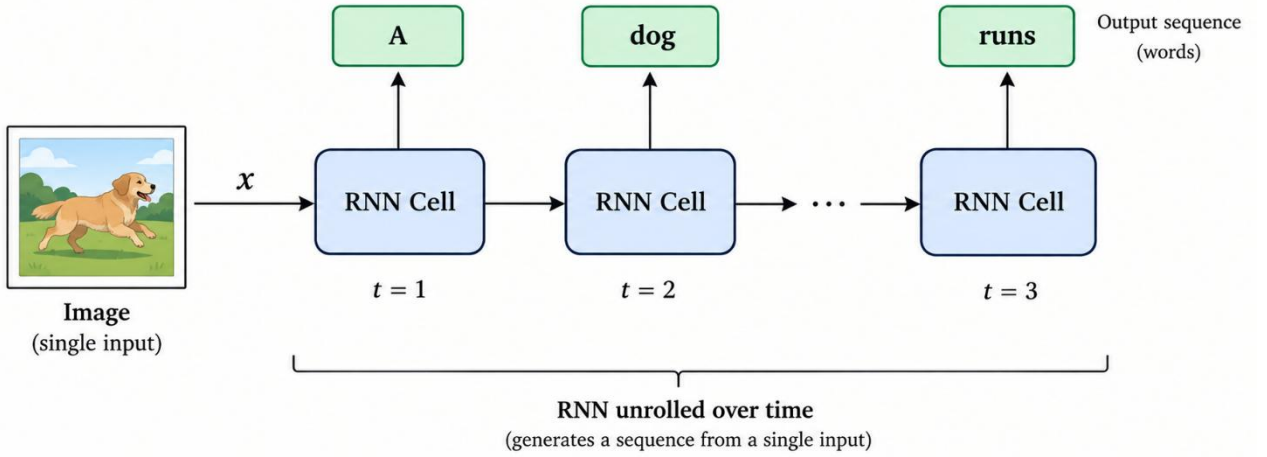
একটি একক ইনপুট থেকে একটি সিকোয়েন্স আউটপুট তৈরি হয়। উদাহরণ: একটি ছবি (single input) থেকে RNN তার বর্ণনামূলক বাক্য (sequence of words) তৈরি করে — Image Captioning।

চিত্র / Figure 4: One-to-Many আর্কিটেকচার

এই ডায়াগ্রামে যা দেখানো হবে (What this diagram should show):

- বামে একটি ছবির আইকন (Image) ইনপুট হিসেবে
- একটি RNN ব্লক থেকে ডানে ধারাবাহিকভাবে কয়েকটি শব্দ-আউটপুট বের হচ্ছে: “A”, “dog”, “runs”
- প্রতিটি আউটপুট পরবর্তী স্টেপের ইনপুটেও ফিরে যাচ্ছে তীরচিহ্ন দিয়ে

One-to-Many RNN (Image Captioning Example)



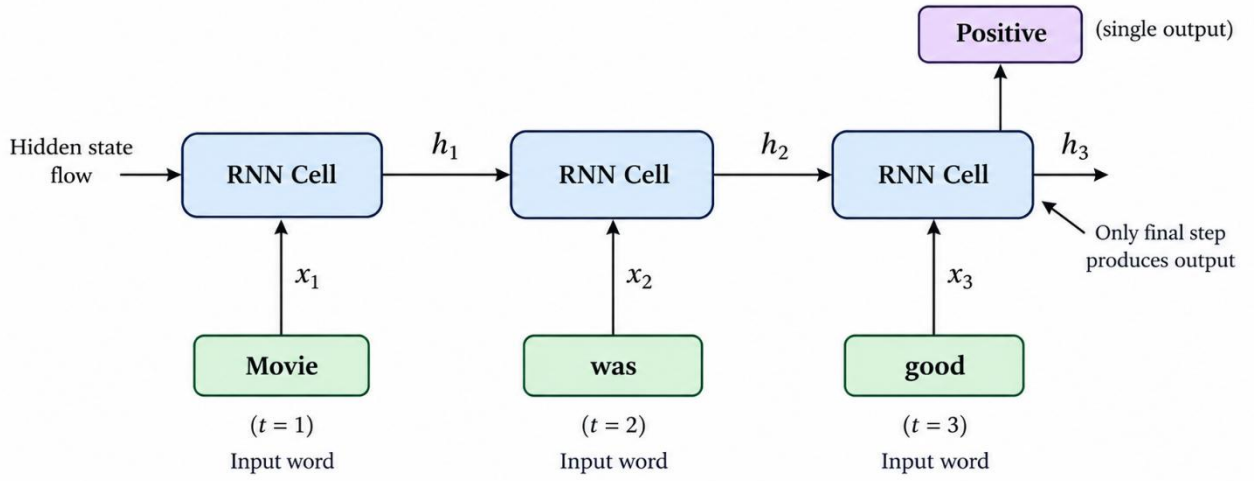
৫.৩ Many-to-One

একটি সিকোয়েন্স ইনপুট থেকে একটি একক আউটপুট তৈরি হয়। উদাহরণ: “Movie was good” — এই রিভিউ পড়ে RNN একটি একক সিদ্ধান্ত দেয় (Positive/Negative) — Sentiment Analysis।

চিত্র / Figure 5: Many-to-One আর্কিটেকচার (Sentiment Analysis)

এই ডায়াগ্রামে যা দেখানো হবে (What this diagram should show):

- তিনটি ইনপুট শব্দ x_1 (“Movie”), x_2 (“was”), x_3 (“good”) তিনটি RNN cell-এ ধারাবাহিকভাবে প্রবেশ করছে
- শুধুমাত্র শেষ cell থেকে একটি একক আউটপুট বের হচ্ছে: “Positive”
- মাঝের cell গুলো থেকে কোনো আউটপুট বের হচ্ছে না, শুধু hidden state ধারাবাহিকভাবে পরের cell-এ যাচ্ছে

Many-to-One RNN (Sentiment Analysis)**৫.৪ Many-to-Many**

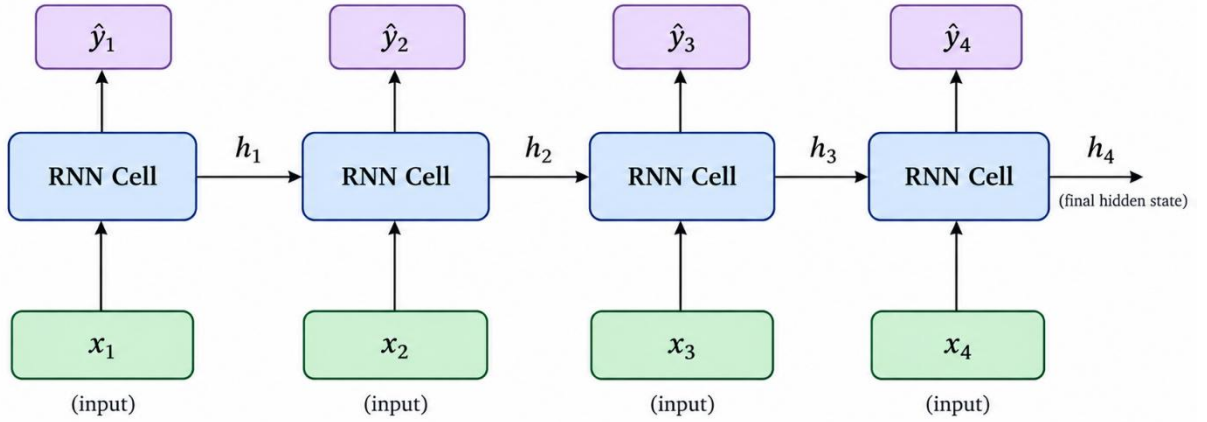
একটি সিকোয়েন্স ইনপুট থেকে একটি সিকোয়েন্স আউটপুট তৈরি হয়। উদাহরণ: Machine Translation (একটি ভাষার বাক্য থেকে অন্য ভাষার বাক্য), Video Frame Classification।

চিত্র / Figure 6: Many-to-Many আর্কিটেকচার**এই ডায়াগ্রামে যা দেখানো হবে (What this diagram should show):**

- একটি সারিতে ৩-৪টি ইনপুট ($x_1...x_4$) ধারাবাহিক RNN cell-এ প্রবেশ করছে
- প্রতিটি cell থেকে একটি করে আউটপুট ($\hat{y}_1...\hat{y}_4$) বের হচ্ছে
- hidden state একটি cell থেকে পরের cell-এ ধারাবাহিকভাবে প্রবাহিত হচ্ছে

Many-to-Many RNN

(Sequence to Sequence / Input and Output of the Same Length)

**Key Takeaways**

- RNN-এর চারটি প্রধান ধরন: One-to-One, One-to-Many, Many-to-One, Many-to-Many।
- আর্কিটেকচারের পছন্দ সমস্যার ধরনের উপর নির্ভর করে — sentiment analysis-এর জন্য Many-to-One, translation-এর জন্য Many-to-Many।
- One-to-One প্রকৃতপক্ষে recurrent নয়, বেসলাইন হিসেবে দেখানো হয়।

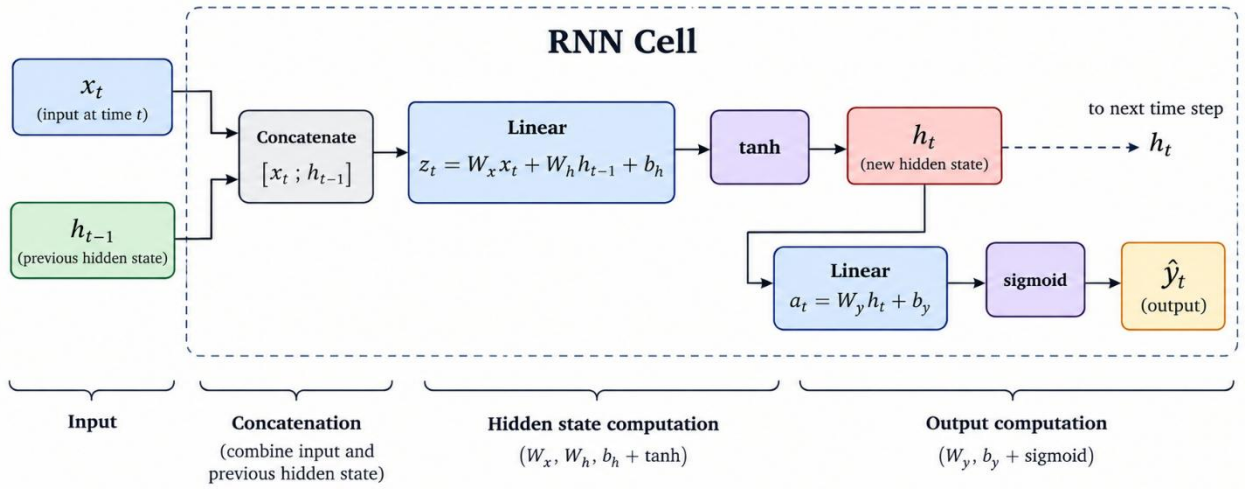
৬. RNN আর্কিটেকচার এবং প্যারামিটার (Architecture & Parameters)**৬.১ RNN Cell**

RNN-এর প্রতিটি time-step-এ একই “cell” পুনরায় ব্যবহৃত হয়। এই cell-টি দুটি ইনপুট নেয় — বর্তমান ইনপুট এবং আগের hidden state — এবং দুটি আউটপুট তৈরি করে: নতুন hidden state এবং (প্রয়োজনে) একটি prediction।

চিত্র / Figure 7: RNN সেলের অভ্যন্তরীণ গঠন (Internal Cell Architecture)

এই ডায়াগ্রামে যা দেখানো হবে (What this diagram should show):

- h_{t-1} এবং x_t একসাথে concatenate হয়ে একটি লিনিয়ার ট্রান্সফর্মেশনে (W_x, W_h, b_h) প্রবেশ করছে
- লিনিয়ার আউটপুট (z_h) একটি tanh activation-এর মধ্য দিয়ে গিয়ে নতুন hidden state h_t তৈরি করছে
- h_t থেকে একটি দ্বিতীয় লিনিয়ার ট্রান্সফর্মেশন (W_y, b_y) হয়ে sigmoid activation দিয়ে আউটপুট $\hat{y}_t$ তৈরি হচ্ছে
- h_t থেকে একটি ডটেড তীর পরবর্তী time-step-এ যাচ্ছে



৬.২ ও ৬.৩ Weight Matrices ও Biases

ধরা যাক, ইনপুটের মাত্রা n_x , hidden state-এর মাত্রা n_h এবং আউটপুটের মাত্রা n_y । তাহলে RNN cell-এর প্যারামিটারগুলোর আকার (dimension) নিম্নরূপ। এই নোটে W_x (Input-to-Hidden Weight) এবং W_h (Hidden-to-Hidden / Recurrent Weight) নোটেশন ব্যবহার করা হয়েছে। কিছু টেক্সটবুকে একই প্যারামিটারকে W_i বা W_{ih} নামেও ডাকা হয় — এই নোটে সামঞ্জস্যের জন্য সবসময় W_x, W_h, W_y ব্যবহার করা হবে।

প্যারামিটার	পূর্ণ নাম / ভূমিকা	মাত্রা (Dimension)
W_x	Input-to-Hidden Weight	$n_h \times n_x$
W_h	Hidden-to-Hidden (Recurrent) Weight	$n_h \times n_h$

প্যারামিটার	পূর্ণ নাম / ভূমিকা	মাত্রা (Dimension)
W_y	Hidden-to-Output Weight	$n_y \times n_h$
b_h	Hidden Layer Bias	$n_h \times 1$
b_y	Output Layer Bias	$n_y \times 1$

৬.৪ Parameter Sharing

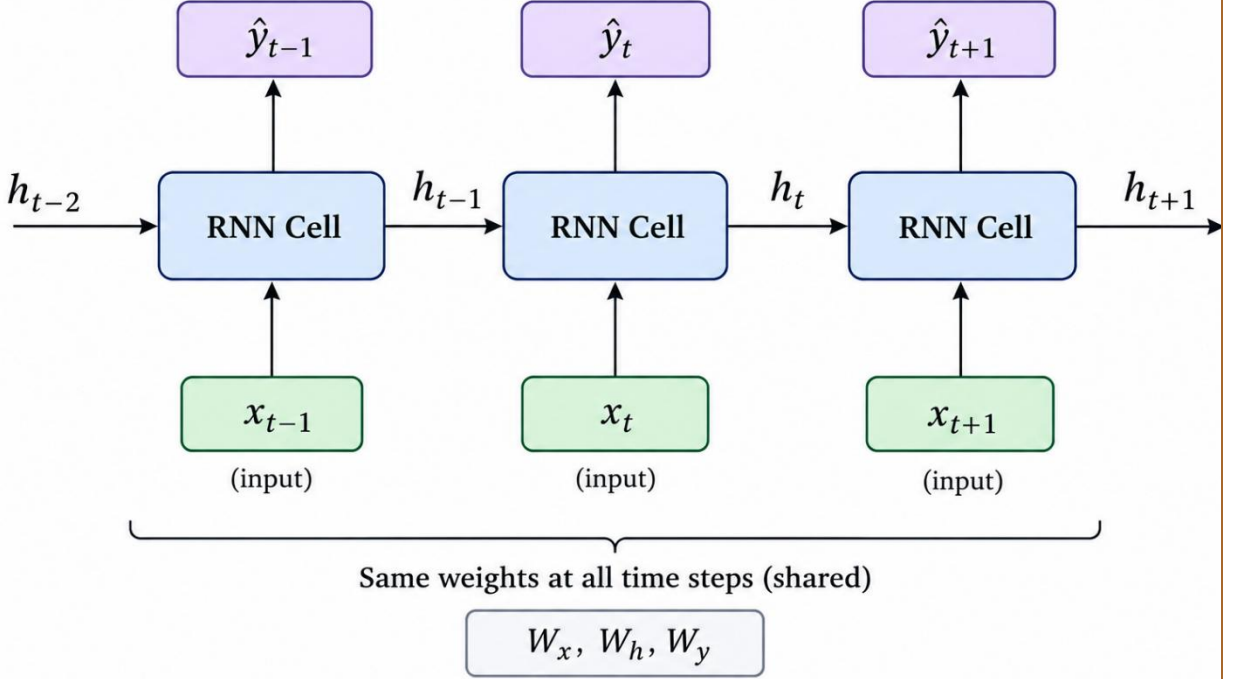
গুরুত্বপূর্ণ পর্যবেক্ষণ (Parameter Sharing)

- প্রতিটি time-step-এ ঠিক W_x , W_h , W_y এবং bias-গুলোর একই কপি পুনরায় ব্যবহার করা হয় (parameter sharing)। এটিই RNN-কে যেকোনো দৈর্ঘ্যের সিকোয়েন্সে কাজ করার ক্ষমতা দেয় — কারণ প্যারামিটারের সংখ্যা সিকোয়েন্স দৈর্ঘ্যের উপর নির্ভর করে না।

চিত্র / Figure 8: Unfolded RNN (সময়ের সাপেক্ষে উন্মোচিত)

এই ডায়াগ্রামে যা দেখানো হবে (What this diagram should show):

- তিনটি সময়-ধাপ $t-1$, t , $t+1$ জুড়ে একই RNN cell তিনবার আঁকা
- h_{t-2} থেকে শুরু করে h_{t-1} , h_t , h_{t+1} পর্যন্ত hidden state ধারাবাহিকভাবে বাম থেকে ডানে প্রবাহিত হচ্ছে
- প্রতিটি cell-এর নিচে তার নিজস্ব ইনপুট x_{t-1} , x_t , x_{t+1} এবং উপরে আউটপুট $\hat{y}_{t-1}$, $\hat{y}_t$, $\hat{y}_{t+1}$
- তিনটি cell-এর নিচে একটি বক্সনী ও ক্যাপশন: “Same W_x , W_h , W_y shared across all time steps”



৬.৫ Dimension Check (মাত্রা যাচাই)

Beginners-দের জন্য matrix multiplication-এর dimension মিলিয়ে দেখা অত্যন্ত গুরুত্বপূর্ণ একটি অভ্যাস। ধরা যাক:

$$\mathbf{x}_t \in \mathbb{R}^{n_x} \text{ এবং } \mathbf{h}_{t-1} \in \mathbb{R}^{n_h}$$

তাহলে ওয়েট ম্যাট্রিক্সগুলোর dimension এমন হতে হবে যাতে matrix multiplication বৈধ (valid) হয়:

$$\mathbf{W}_x \in \mathbb{R}^{n_h \times n_x}, \mathbf{W}_h \in \mathbb{R}^{n_h \times n_h}, \mathbf{b}_h \in \mathbb{R}^{n_h}, \mathbf{W}_y \in \mathbb{R}^{n_y \times n_h}$$

গুণনের dimension মেলানো: একটি $(n_h \times n_x)$ ম্যাট্রিক্সকে একটি $(n_x \times 1)$ ভেক্টর দিয়ে গুণ করলে ফলাফল হয় $(n_h \times 1)$:

$$(n_h \times n_x) \cdot (n_x \times 1) = (n_h \times 1)$$

অর্থাৎ ভেতরের দুটি সংখ্যা (n_x) একে অপরের সাথে বাতিল (cancel) হয়ে যায়, এবং বাইরের দুটি সংখ্যা (n_h এবং 1) চূড়ান্ত ফলাফলের আকার নির্ধারণ করে। এই একই নিয়ম $\mathbf{W}_h \mathbf{h}_{t-1}$ এবং $\mathbf{W}_y \mathbf{h}_t$ -এর জন্যও প্রযোজ্য।

Common Mistakes

- $\mathbf{W}_x$ এবং $\mathbf{W}_h$ -এর dimension এক ভেবে ভুল করা — $\mathbf{W}_x$ -এর আকার n_x -এর উপর নির্ভর করে, কিন্তু $\mathbf{W}_h$ সবসময় $(n_h \times n_h)$ — square matrix, কারণ এটি hidden state-কে hidden state-এ ম্যাপ করে।
- প্রতিটি time-step-এ নতুন ওয়েট ম্যাট্রিক্স তৈরি হয় ভাবা — বাস্তবে একই ওয়েট বারবার ব্যবহৃত হয় (parameter sharing)।

Key Takeaways

- RNN cell-এর তিনটি মূল ওয়েট ম্যাট্রিক্স: $\mathbf{W}_x$ (input→hidden), $\mathbf{W}_h$ (hidden→hidden), $\mathbf{W}_y$ (hidden→output)।
- Parameter Sharing-এর কারণে প্যারামিটার সংখ্যা সিকোয়েন্সের দৈর্ঘ্যের উপর নির্ভর করে না।
- Dimension check সবসময় (বাইরের সংখ্যা $\times$ বাইরের সংখ্যা) নিয়মে যাচাই করা উচিত।

৭. RNN Forward Propagation

প্রতিটি time-step t -এ RNN নিম্নলিখিত তিনটি ধাপ ধারাবাহিকভাবে সম্পন্ন করে। এই নোট জুড়ে ইনপুটের জন্য $\mathbf{x}_t$, hidden state-এর জন্য $\mathbf{h}_t$, hidden pre-activation-এর জন্য $\mathbf{z}_t$, এবং predicted output-এর জন্য $\hat{y}_t$ — এই একই নোটেশন সামঞ্জস্যপূর্ণভাবে ব্যবহার করা হয়েছে।

৭.১ হিডেন লেয়ারের Pre-activation (Net Input)

$$\mathbf{z}_{h,t} = \mathbf{x}_t \mathbf{W}_x + \mathbf{h}_{t-1} \mathbf{W}_h + \mathbf{b}_h$$

এখানে $\mathbf{W}_x \mathbf{x}_t$: বর্তমান ইনপুট থেকে অবদান; $\mathbf{W}_h \mathbf{h}_{t-1}$: আগের memory থেকে অবদান; $\mathbf{b}_h$: bias। দুটি অবদান যোগ হয়ে একটি “net pre-activation” তৈরি করে।

৭.২ Hidden State — অ-রৈখিক সক্রিয়করণ

$$\mathbf{h}_t = \tanh(\mathbf{z}_{h,t})$$

$\tanh$ ফাংশনটি $(-1, 1)$ রেঞ্জে আউটপুট সংকুচিত (squash) করে, যা hidden state-কে সংখ্যাগতভাবে স্থিতিশীল (numerically stable) রাখে এবং ধনাত্মক-ঋণাত্মক উভয় দিকের তথ্য বহন করতে দেয়।

৭.৩ আউটপুট লেয়ার

$$\hat{y}_t = \sigma(\mathbf{h}_t \mathbf{W}_y + \mathbf{b}_y)$$

Binary classification-এর ক্ষেত্রে $\sigma = \text{sigmoid}$ ব্যবহৃত হয়; Multi-class classification-এর ক্ষেত্রে সাধারণত softmax ব্যবহার করা হয়।

৭.৪ Loss

Loss ফাংশন নির্বাচন সমস্যার ধরনের উপর নির্ভর করে — Binary classification-এর জন্য Binary Cross-Entropy (BCE), Multi-class-এর জন্য Categorical Cross-Entropy। এই নোটের numerical উদাহরণে BCE ব্যবহার করা হয়েছে, যা পরবর্তী সেকশনে বিস্তারিত দেখানো হবে।

Key Takeaways

- Forward propagation তিনটি ধাপে হয়: pre-activation $\rightarrow$ hidden state ($\tanh$) $\rightarrow$ output (sigmoid/softmax)।
- $\mathbf{z}_{h,t} = \mathbf{x}_t \mathbf{W}_x + \mathbf{h}_{t-1} \mathbf{W}_h + \mathbf{b}_h$ — বর্তমান ইনপুট ও অতীত memory উভয়ের সমষ্টি।
- একই নোটেশন $(\mathbf{x}_t, \mathbf{h}_t, \mathbf{z}_t, \hat{y}_t)$ পুরো নোট জুড়ে সামঞ্জস্যপূর্ণভাবে ব্যবহৃত হয়েছে।

৮. সম্পূর্ণ সাংখ্যিক উদাহরণ (Complete Numerical Example)

এই সেকশনে একটি ছোট, দুই time-step-এর RNN-এর সম্পূর্ণ forward pass, loss গণনা, এবং BPTT হাতে-কলমে (hand-worked) দেখানো হয়েছে। শেখার সুবিধার্থে এখানে

ইচ্ছাকৃতভাবে **scalar (একক-মাত্রিক) weight** ব্যবহার করা হয়েছে, যাতে ভেক্টর/ম্যাট্রিক্স নোটেশনের জটিলতা ছাড়াই গণিতটি সম্পূর্ণ স্বচ্ছ (transparent) থাকে এবং প্রতিটি ধাপ হাতে-কলমে যাচাইযোগ্য হয়। সাধারণ (general) ক্ষেত্রে এই একই সমীকরণগুলো ভেক্টর/ম্যাট্রিক্স আকারে প্রযোজ্য, যেমনটি সেকশন ৭-এ দেখানো হয়েছে।

৮.১ প্রদত্ত মান (Given Values)

চলক (Variable)	মান (Value)	অর্থ
x_1, x_2	1.0, 2.0	দুটি time-step-এর ইনপুট
h_0	0	প্রাথমিক hidden state
y	1	প্রকৃত (true) লেবেল
W_x	0.5	Input-to-Hidden weight
W_h	0.8	Hidden-to-Hidden weight
W_y	1.0	Hidden-to-Output weight
b_h, b_y	0, 0	বায়াস (bias)
η (learning rate)	0.1	Gradient Descent step size

৮.২ Forward Pass at $t = 1$

$$z_{h,1} = x_1 W_x + h_0 W_h + b_h$$

$$z_{h,1} = (1.0)(0.5) + (0)(0.8) + 0 = 0.5$$

$$h_1 = \tanh(0.5) \approx 0.4621$$

৮.৩ Forward Pass at $t = 2$

$$z_{h,2} = x_2 W_x + h_1 W_h + b_h$$

$$z_{h,2} = (2.0)(0.5) + (0.4621)(0.8) + 0 \approx 1.3697$$

$$h_2 = \tanh(1.3697) \approx 0.8786$$

৮.৪ আউটপুট (Output)

$$z_o = h_2 W_y + b_y = (0.8786)(1.0) + 0 = 0.8786$$

$$\hat{y} = \sigma(0.8786) = \frac{1}{1 + e^{-0.8786}} \approx 0.7065$$

৮.৫ Binary Cross-Entropy (BCE) Loss

$$L = -[y \ln(\hat{y}) + (1 - y) \ln(1 - \hat{y})]$$

যেহেতু $y = 1$, দ্বিতীয় পদটি $(1 - y) = 0$ হওয়ায় শূন্য হয়ে যায়:

$$L = -\ln(0.7065) \approx 0.3474$$

গুরুত্বপূর্ণ পর্যবেক্ষণ

- BCE সমীকরণের শুরুর ঋণাত্মক চিহ্ন $(-)$ কখনোই বাদ দেওয়া যাবে না $-\log(\hat{y})$ সবসময় ঋণাত্মক (যেহেতু $0 < \hat{y} < 1$), তাই ঋণাত্মক চিহ্ন ছাড়া Loss নিজেই ঋণাত্মক হয়ে যাবে, যা অর্থহীন।

৮.৬ BPTT — Backpropagation Through Time

ধাপ ১ — আউটপুট গ্রেডিয়েন্ট: Sigmoid + BCE combo-র একটি সুন্দর বৈশিষ্ট্য হলো, এর গ্রেডিয়েন্ট সরলীকৃত রূপে পাওয়া যায়:

$$\frac{\partial L}{\partial z_o} = \hat{y} - y$$

$$\frac{\partial L}{\partial z_o} = 0.7065 - 1 = -0.2935$$

ধাপ ২ — W_y -এর গ্রেডিয়েন্ট:

$$\frac{\partial L}{\partial W_y} = \frac{\partial L}{\partial z_o} \cdot h_2 = (-0.2935)(0.8786) \approx -0.2579$$

ধাপ ৩ — $t = 2$ -এ ফিরে আসা (h_2 পর্যন্ত):

$$\frac{\partial L}{\partial h_2} = \frac{\partial L}{\partial z_o} \cdot W_y = (-0.2935)(1.0) = -0.2935$$

এবার $\tanh$ -এর ডেরিভেটিভ প্রয়োগ করতে হবে: $\tanh'(z) = 1 - \tanh^2(z) = 1 - h^2$, $t = 2$ -এ:

$$\tanh'(z_{h,2}) = 1 - (0.8786)^2 \approx 0.2280$$

$$\delta z_{,2} = \frac{\partial L}{\partial h_2} \cdot \tanh'(z_{h,2}) = (-0.2935)(0.2280) \approx -0.0669$$

ধাপ ৪ — $t = 2$ থেকে W_x ও W_h -এ অবদান:

$$(\partial L / \partial W_x)_{(t=2)} = \delta z_{,2} \cdot x_2 = (-0.0669)(2.0) \approx -0.1338$$

$$(\partial L / \partial W_h)_{(t=2)} = \delta z_{,2} \cdot h_1 = (-0.0669)(0.4621) \approx -0.0309$$

ধাপ ৫ – $t = 1$ -এ ফিরে আসা (h_1 পর্যন্ত, BPTT-র মূল ধাপ):

যেহেতু h_2 , W_h -এর মাধ্যমে h_1 -এর উপর নির্ভরশীল ছিল, গ্রেডিয়েন্ট এখন $t = 1$ -এ ফিরে প্রবাহিত হবে:

$$\frac{\partial L}{\partial h_1} = \delta z_{,2} \cdot W_h = (-0.0669)(0.8) \approx -0.0535$$

$$\tanh'(z_{h,1}) = 1 - (0.4621)^2 \approx 0.7864$$

$$\delta z_{,1} = \frac{\partial L}{\partial h_1} \cdot \tanh'(z_{h,1}) = (-0.0535)(0.7864) \approx -0.0421$$

ধাপ ৬ – $t = 1$ থেকে W_x ও W_h -এ অবদান:

$$(\partial L / \partial W_x)_{(t=1)} = \delta z_{,1} \cdot x_1 = (-0.0421)(1.0) \approx -0.0421$$

$$(\partial L / \partial W_h)_{(t=1)} = \delta z_{,1} \cdot h_0 = (-0.0421)(0) = 0$$

Observation

- $(\partial L / \partial W_h)$ at $t=1$ ঠিক শূন্য হওয়ার কারণ $h_0 = 0$ – এটি একটি বিশেষ (coincidental) ফলাফল। এই নির্দিষ্ট উদাহরণের জন্য, সাধারণভাবে সবসময় শূন্য হবে না।

৮.৭ গ্রেডিয়েন্ট সঞ্চয়ন (Gradient Accumulation)

যেহেতু W_x এবং W_h প্রতিটি time-step-এ পুনঃব্যবহৃত হয় (parameter sharing), তাদের চূড়ান্ত গ্রেডিয়েন্ট হলো সমস্ত time-step থেকে প্রাপ্ত আংশিক গ্রেডিয়েন্টের সমষ্টি – এটিই BPTT-র কেন্দ্রীয় ধারণা:

$$\frac{\partial L}{\partial W_x} = \sum_t (\partial L_t / \partial W_x)$$

$$\frac{\partial L}{\partial W_x} = (-0.1338) + (-0.0421) = -0.1759$$

$$\frac{\partial L}{\partial W_h} = (-0.0309) + (0) = -0.0309$$

b_h -এর গ্রেডিয়েন্টও একইভাবে সবগুলো time-step-এর δz -এর সমষ্টি, এবং b_y , W_y শুধুমাত্র আউটপুট layer-এ একবারই ব্যবহৃত হওয়ায় তাদের গ্রেডিয়েন্ট সঞ্চয়নের প্রয়োজন নেই:

$$\frac{\partial L}{\partial b_h} = \delta z_{,1} + \delta z_{,2} = (-0.0421) + (-0.0669) = -0.1090$$

$$\frac{\partial L}{\partial b_y} = \partial L / \partial z_o = -0.2935$$

প্যারামিটার	চূড়ান্ত গ্রেডিয়েন্ট ($\partial L / \partial \theta$)
W_x	-0.1759
W_h	-0.0309
W_y	-0.2579
b_h	-0.1090
b_y	-0.2935

৮.৮ Gradient Descent — প্যারামিটার আপডেট

$$\theta_{new} = \theta_{old} - \eta \cdot (\partial L / \partial \theta)$$

$\eta = 0.1$ বসিয়ে প্রতিটি প্যারামিটার আলাদাভাবে আপডেট করা যায়:

প্যারামিটার	পুরনো মান	আপডেট ($-\eta \times$ গ্রেডিয়েন্ট)	নতুন মান
W_x	0.5000	$-(0.1)(-0.1759) = +0.0176$	0.5176
W_h	0.8000	$-(0.1)(-0.0309) = +0.0031$	0.8031
W_y	1.0000	$-(0.1)(-0.2579) = +0.0258$	1.0258
b_h	0.0000	$-(0.1)(-0.1090) = +0.0109$	0.0109
b_y	0.0000	$-(0.1)(-0.2935) = +0.0293$	0.0293

৮.৯ যাচাইকরণ (Verification)

লক্ষণীয় যে, সবগুলো গ্রেডিয়েন্টই ঋণাত্মক (negative), তাই Gradient Descent-এর – চিহ্নের কারণে সবগুলো প্যারামিটারই বৃদ্ধি পেয়েছে (increased)। এটি প্রত্যাশিত ফলাফল, কারণ $y = 1$ হওয়া সত্ত্বেও মডেলের প্রাথমিক প্রেডিকশন ছিল $\hat{y} \approx 0.7065$ — অর্থাৎ মডেলটিকে আরও বেশি “confident” হতে হবে ($\hat{y}$ -কে 1-এর কাছাকাছি নিতে হবে)। নতুন প্যারামিটারগুলো দিয়ে পুনরায় forward pass চালালে $\hat{y}$ -এর মান বেড়ে 1-এর দিকে সরে যাবে এবং Loss কিছুটা কমে যাবে — যা যাচাই করে যে গ্রেডিয়েন্ট ও আপডেটের দিক (sign) সঠিক।

Key Takeaways

- Forward pass: $z_{h,t} \rightarrow h_t = \tanh(z_{h,t}) \rightarrow z_o \rightarrow \hat{y} = \sigma(z_o) \rightarrow L \text{ (BCE)}$ ।
- BPTT গ্রেডিয়েন্ট আউটপুট থেকে শুরু করে সময়ের বিপরীতে ($t=2 \rightarrow t=1$) প্রবাহিত হয়।
- Shared parameter (W_x, W_h)-এর গ্রেডিয়েন্ট হলো সব time-step-এর আংশিক গ্রেডিয়েন্টের যোগফল।
- সব গ্রেডিয়েন্ট ঋণাত্মক হওয়ায় Gradient Descent-এর পর সব প্যারামিটার বৃদ্ধি পেয়েছে — Loss কমানোর দিকেই।

Common Mistakes

- Gradient-কে Updated Parameter-এর সাথে গুলিয়ে ফেলা — গ্রেডিয়েন্ট হলো দিক-নির্দেশক পরিমাণ, চূড়ান্ত মান নয়।
- BCE সমীকরণের ঋণাত্মক চিহ্ন ভুলে যাওয়া।
- মনে করা যে প্রতিটি time-step-এর গ্রেডিয়েন্ট আলাদা আলাদা প্যারামিটারের জন্য প্রযোজ্য — বাস্তবে shared parameter-এর গ্রেডিয়েন্ট সবসময় accumulate (যোগ) হয়।

৯. Backpropagation Through Time (BPTT)

৯.১ কেন সাধারণ Backpropagation যথেষ্ট নয়?

সাধারণ feed-forward নেটওয়ার্কে backpropagation একবার, layer-by-layer করা হয় — কোনো recurrent connection থাকে না। কিন্তু RNN-এ hidden state h_t নিজের আগের মান h_{t-1} -এর উপর নির্ভরশীল, যা আবার h_{t-2} -এর উপর নির্ভরশীল — এভাবে একটি “chain” তৈরি হয় যা সময়ের মধ্য দিয়ে বিস্তৃত। সাধারণ backpropagation এই recurrent নির্ভরতা ধরতে পারে না, তাই RNN-কে প্রথমে সময়ের সাপেক্ষে “unfold” বা “unroll” করে একটি গভীর, ফিড-ফরোয়ার্ড-সদৃশ নেটওয়ার্কে রূপান্তর করা হয়, তারপর সেই unfolded নেটওয়ার্কে সাধারণ backpropagation প্রয়োগ করা হয়।

সংজ্ঞা

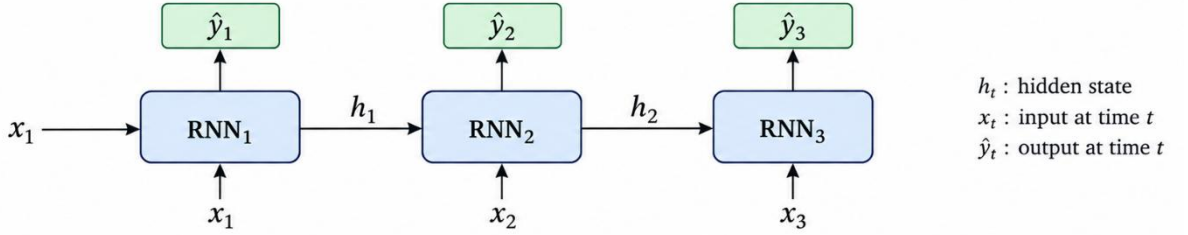
- Backpropagation Through Time (BPTT) = RNN-কে সময়ের সাপেক্ষে unroll করার পর তার উপর backpropagation প্রয়োগ করা।

চিত্র / Figure 9: BPTT — Forward ও Backward প্রবাহ

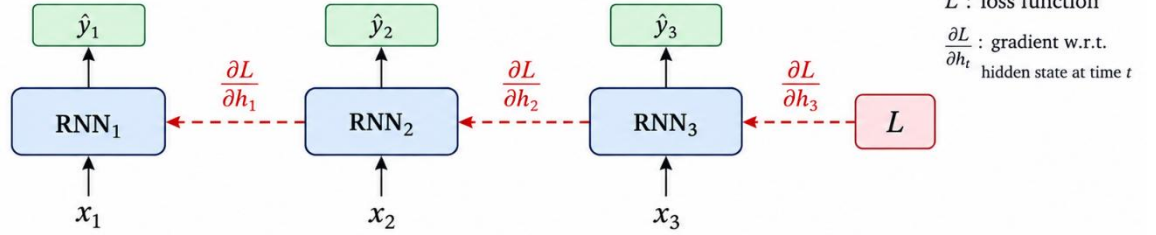
এই ডায়াগ্রামে যা দেখানো হবে (What this diagram should show):

- উপরের সারিতে forward direction: $x_1 \rightarrow \text{RNN}_1 \rightarrow \text{RNN}_2 \rightarrow \text{RNN}_3$, বাম থেকে ডানে তীরচিহ্ন
- নিচের সারিতে backward direction: $L \rightarrow \text{RNN}_3 \rightarrow \text{RNN}_2 \rightarrow \text{RNN}_1$, ডান থেকে বামে লাল/ড্যাশড তীরচিহ্ন
- স্পষ্টভাবে লেবেল করা: “Forward Pass” এবং “Backpropagation Through Time (gradient flow)”

Forward Pass (left to right)



Backpropagation Through Time (gradient flow)



৯.২ Chain Rule-এর ভূমিকা

BPTT সম্পূর্ণভাবে chain rule-এর উপর ভিত্তি করে তৈরি। প্রতিটি প্যারামিটারের গ্রেডিয়েন্ট বের করতে হলে Loss থেকে শুরু করে সেই প্যারামিটার পর্যন্ত পৌঁছানোর সমস্ত সম্ভাব্য “পথ” (path) ধরে গ্রেডিয়েন্ট গুণ করে এগোতে হয়। সেকশন ৮-এর সাংখ্যিক উদাহরণে ঠিক এই প্রক্রিয়াটিই হাতে-কলমে দেখানো হয়েছে।

৯.৩ Temporal Gradient Flow

দুই ধরনের গ্রেডিয়েন্ট আলাদা করে বোঝা জরুরি:

ধরন	সংজ্ঞা
Local Gradient	একটি নির্দিষ্ট time-step-এর ভেতরে হিসাব করা গ্রেডিয়েন্ট (যেমন δz_t)
Temporal Gradient	একটি পরবর্তী time-step থেকে আগের time-step-এ প্রবাহিত (propagated) গ্রেডিয়েন্ট

সেকশন ৮-এ দেখা গেছে, $t = 2$ -এর গ্রেডিয়েন্ট $(\delta z, 2)$ W_h -এর মাধ্যমে $t = 1$ -এ $(\partial L / \partial h_1)$ হয়ে প্রবাহিত হয়েছিল — এটিই temporal gradient flow-এর একটি বাস্তব উদাহরণ।

৯.৪ Shared Parameter Gradient

যেহেতু W_x, W_h, b_h প্রতিটি time-step-এ একই থাকে (parameter sharing), তাদের প্রকৃত গ্রেডিয়েন্ট বের করতে সমস্ত time-step-এর আংশিক অবদান যোগ করতে হয়:

$$\frac{\partial L}{\partial W_h} = \sum_t (\partial L_t / \partial W_h)$$

গুরুত্বপূর্ণ পর্যবেক্ষণ

- এই accumulation (যোগফল) ধাপটিই BPTT-কে সাধারণ backpropagation থেকে আলাদা করে। একটি single shared parameter একাধিক time-step-এ ব্যবহৃত হওয়ায়, তার সামগ্রিক প্রভাব বোঝার জন্য প্রতিটি ব্যবহারের গ্রেডিয়েন্ট আলাদাভাবে হিসাব করে যোগ করতে হয় — বাদ দিলে গ্রেডিয়েন্ট ভুল হবে।

Key Takeaways

- BPTT = RNN unroll করে সাধারণ backpropagation প্রয়োগ করা।
- Chain Rule-ই BPTT-র মূল গাণিতিক ভিত্তি — Loss থেকে প্রতিটি প্যারামিটার পর্যন্ত পথ ধরে গ্রেডিয়েন্ট গুণ হয়।
- Temporal Gradient Flow-এর মাধ্যমে গ্রেডিয়েন্ট পরবর্তী time-step থেকে আগের time-step-এ প্রবাহিত হয়।
- Shared parameter-এর গ্রেডিয়েন্ট = সব time-step-এর আংশিক গ্রেডিয়েন্টের সমষ্টি।

১০. গ্রেডিয়েন্ট সমস্যা (Gradient Problems)

১০.১ Vanishing Gradient

স্বজ্ঞামূলক ব্যাখ্যা (Intuitive Explanation): যখন একটি সিকোয়েন্স দীর্ঘ হয়, BPTT-র মাধ্যমে গ্রেডিয়েন্ট অনেকগুলো time-step পেরিয়ে পেছনে যেতে হয়। প্রতিটি step-এ গ্রেডিয়েন্ট W_h এবং $\tanh'(z)$ -এর একটি factor দ্বারা গুণিত হয়। যদি এই factor-গুলো বারবার 1-এর চেয়ে ছোট হয়, তাহলে বহুবার গুণনের ফলে গ্রেডিয়েন্টের মান দ্রুত সংকুচিত হয়ে প্রায় শূন্যে পরিণত হয় — ফলে প্রথম দিকের time-step গুলো কার্যত কোনো learning signal পায় না।

গাণিতিক রূপ (Mathematical Form): সরলীকৃত আকারে, k সংখ্যক time-step পেছনের গ্রেডিয়েন্ট আনুমানিকভাবে লেখা যায়:

$$\frac{\partial L}{\partial h_{(t-k)}} \approx \frac{\partial L}{\partial h_t} \cdot \prod_{j=1}^k (W_h \cdot \tanh'(z_j))$$

গুরুত্বপূর্ণ – নির্ভুলতা বজায় রাখা

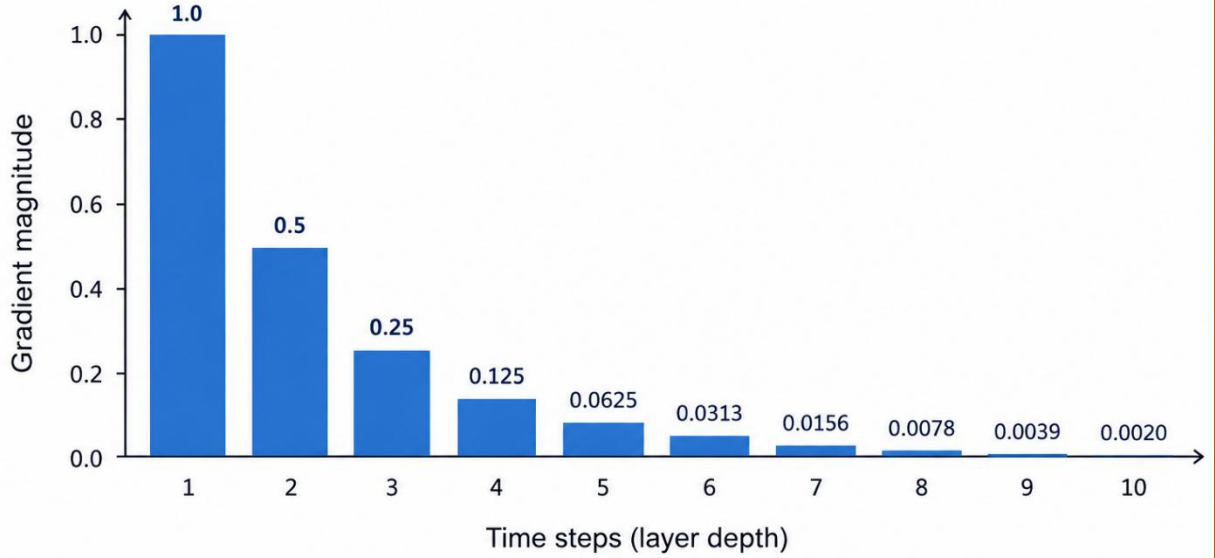
- এটি বলা ভাল হবে যে $\tanh'(z)$ সবসময় ০ থেকে ০.২৫-এর মধ্যে থাকে (এই বিভ্রান্তি sigmoid-এর সাথে গুলিয়ে ফেলার কারণে হয়)। প্রকৃতপক্ষে $\tanh'(z) = 1 - \tanh^2(z)$ -এর সর্বোচ্চ মান হলো ১, যা $z = 0$ -এ ঘটে। শুধুমাত্র saturated অঞ্চলে ($|z|$ বড়) $\tanh'(z)$ ছোট হয়ে যায়। তাই vanishing gradient সমস্যাটি $\tanh'(z)$ -এর কোনো নির্দিষ্ট সীমার কারণে নয়, বরং $|W_h \cdot \tanh'(z_j)|$ বারবার ১-এর চেয়ে ছোট হলে তাদের ক্রমাগত গুণনের প্রভাবে ঘটে।

চিত্র / Figure 10: Vanishing Gradient

এই ডায়াগ্রামে যা দেখানো হবে (What this diagram should show):

- একটি অনুভূমিক সংখ্যারেখায় বাম থেকে ডানে ক্রমহ্রাসমান বার/বাক্স: $1.0 \rightarrow 0.5 \rightarrow 0.25 \rightarrow 0.125 \rightarrow \dots$
- প্রতিটি বারের উচ্চতা আগেরটির অর্ধেক, চোখে দেখা যায় এমনভাবে দ্রুত সংকুচিত হচ্ছে
- নিচে ক্যাপশন: “Early time steps receive almost no gradient signal”
- একটি ছোট তীরচিহ্ন time-step অক্ষ বরাবর $t=1$ থেকে $t=T$ পর্যন্ত দেখানো

Vanishing Gradient



Early time steps receive almost no gradient signal

১০.২ Exploding Gradient

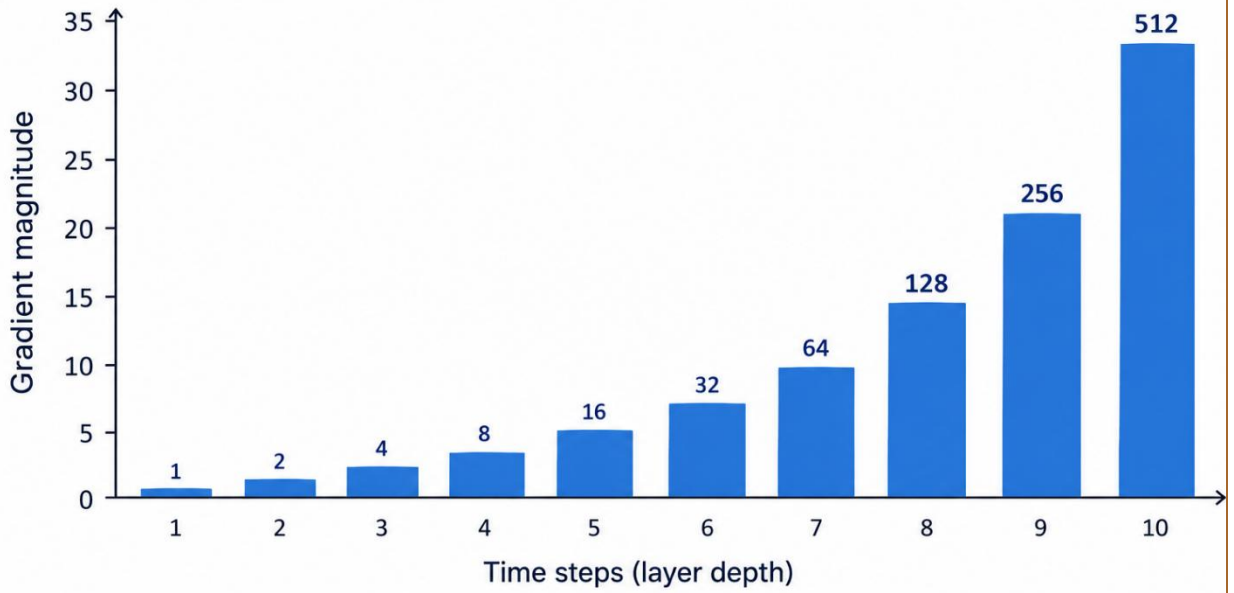
বিপরীত পরিস্থিতিতে, যদি $|W_h \cdot \tanh'(z_j)|$ বারবার 1-এর চেয়ে বড় হয়, তাহলে ক্রমাগত গুণনের ফলে গ্রেডিয়েন্টের মান exponentially বৃদ্ধি পেয়ে অত্যন্ত বড় (এমনকি numerically unstable / NaN) হয়ে যেতে পারে।

চিত্র / Figure 11: Exploding Gradient

এই ডায়াগ্রামে যা দেখানো হবে (What this diagram should show):

- একটি অনুভূমিক সংখ্যারেখায় বাম থেকে ডানে ক্রমবর্ধমান বার/বাক্স: $1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 16 \rightarrow \dots$
- প্রতিটি বারের উচ্চতা আগেরটির দ্বিগুণ, দ্রুত বৃদ্ধি দৃশ্যমান
- নিচে ক্যাপশন: “Gradients can grow exponentially large and destabilize training”

Exploding Gradient



Gradients can grow exponentially large and destabilize training.

১০.৩ Gradient Clipping

Exploding gradient-এর সবচেয়ে ব্যবহারিক সমাধান হলো Gradient Clipping — গ্রেডিয়েন্ট ভেক্টরের norm ($\|g\|$) যদি একটি নির্দিষ্ট threshold τ ছাড়িয়ে যায়, তাহলে তাকে স্কেল-ডাউন করে সেই সীমার মধ্যে আনা হয়:

$$\text{if } \|g\| > \tau, \quad g \leftarrow g \cdot (\tau / \|g\|)$$

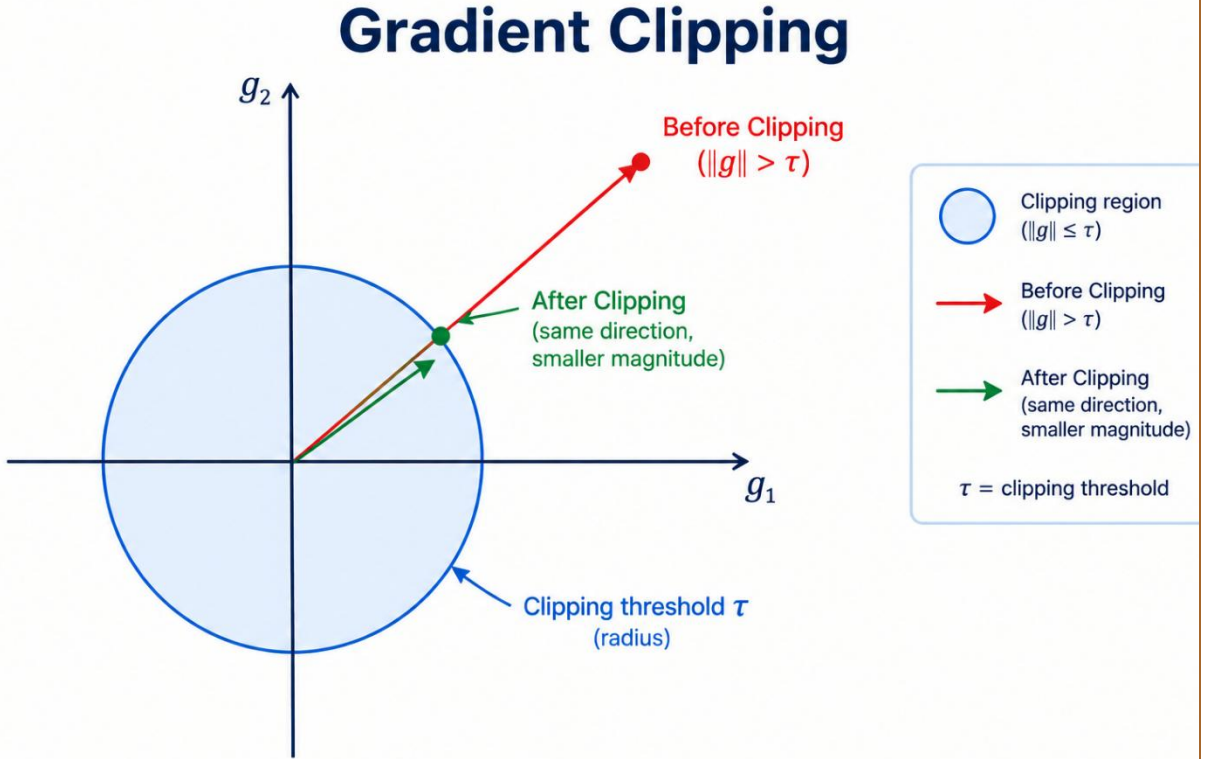
- গ্রেডিয়েন্টের দিক (direction) অপরিবর্তিত থাকে — শুধু মান (magnitude) সীমিত হয়।
- প্যারামিটার আপডেটের গতি (step size) অত্যধিক বড় হওয়া থেকে রক্ষা পায়।

- Training loop numerically stable থাকে, বিশেষত দীর্ঘ সিকোয়েন্সের ক্ষেত্রে।

চিত্র / Figure 12: Gradient Clipping

এই ডায়াগ্রামে যা দেখানো হবে (What this diagram should show):

- বামে একটি দীর্ঘ তীরচিহ্ন (vector), লেবেল “Before Clipping — $\|g\|$ large”
- ডানে একটি ছোট তীরচিহ্ন একই দিকে, লেবেল “After Clipping — same direction, smaller magnitude”
- একটি বৃত্ত (circle) দিয়ে threshold τ -এর সীমা দেখানো



Common Mistakes

- $\tanh'(z)$ সবসময় ০ থেকে ০.২৫-এর মধ্যে থাকে ভাবা — এই সীমা sigmoid-এর ডেরিভেটিভের জন্য প্রযোজ্য, $\tanh$ -এর জন্য নয় ($\tanh'(0) = 1$)।
- Gradient Clipping-কে Vanishing Gradient-এর সমাধান ভাবা — এটি শুধুমাত্র Exploding Gradient-এর জন্য কার্যকর।
- মনে করা যে exploding gradient মানেই মডেল ভুল — এটি numerical instability-র একটি লক্ষণ, সমাধান আছে (clipping)।

Key Takeaways

- Vanishing Gradient: বারবার ছোট factor-এর গুণনে গ্রেডিয়েন্ট দ্রুত শূন্যের কাছাকাছি চলে যায়, প্রথম দিকের time-step শেখা কঠিন হয়।
- Exploding Gradient: বারবার বড় factor-এর গুণনে গ্রেডিয়েন্ট exponentially বৃদ্ধি পায়, training অস্থির হয়ে পড়ে।
- $\tanh'(z)$ -এর সর্বোচ্চ মান 1 ($z=0$ -এ), সবসময় ছোট নয় — এই সূক্ষ্মতা মনে রাখা জরুরি।
- Gradient Clipping দিক ঠিক রেখে শুধু মান সীমিত করে — Exploding Gradient-এর ব্যবহারিক সমাধান।
- এই দুটি সমস্যাই পরবর্তী সেকশনে LSTM প্রবর্তনের মূল প্রেরণা।

১১. Long Short-Term Memory (LSTM)

১১.১ LSTM-এর প্রয়োজনীয়তা (Motivation)

সেকশন ১০-এ দেখা গেছে, Vanilla RNN দীর্ঘ সিকোয়েন্সে long-term dependency শিখতে সংগ্রাম করে — কারণ গ্রেডিয়েন্ট বহু time-step পেরিয়ে প্রবাহিত হওয়ার সময় vanish করে যায়। ১৯৯৭ সালে Hochreiter ও Schmidhuber এই সমস্যা সমাধানের জন্য LSTM প্রবর্তন করেন, যার মূল ধারণা হলো — hidden state ছাড়াও একটি পৃথক, তুলনামূলক “সরাসরি” (direct) তথ্য-প্রবাহের পথ তৈরি করা, যাকে Cell State বলা হয়।

১১.২ Cell State

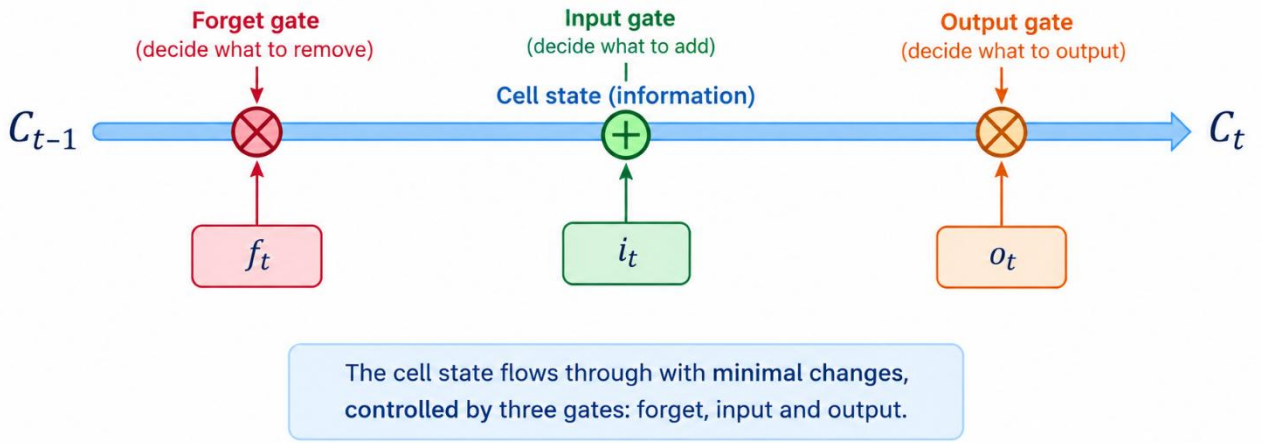
Cell State (C_t) হলো LSTM-এর “দীর্ঘমেয়াদি মেমোরি” — এটি একটি তথ্য-হাইওয়ে (information highway) হিসেবে কাজ করে, যার মধ্য দিয়ে তথ্য প্রায় অপরিবর্তিতভাবে বহুদূর পর্যন্ত প্রবাহিত হতে পারে। প্রতিটি time-step-এ তিনটি “Gate” — Forget, Input, ও Output — নিয়ন্ত্রণ করে ঠিক কতটুকু পুরনো তথ্য রাখা হবে, কতটুকু নতুন তথ্য যোগ হবে, এবং কতটুকু তথ্য বর্তমান আউটপুটে প্রকাশ পাবে।

চিত্র / Figure 13: LSTM Information Highway

এই ডায়াগ্রামে যা দেখানো হবে (What this diagram should show):

- একটি অনুভূমিক রেখা (Cell State, $C_{t-1} \rightarrow C_t$) সরাসরি বাম থেকে ডানে প্রবাহিত হচ্ছে, ন্যূনতম বাধায়
- এই রেখার উপরে তিনটি ছোট ভান্স/গেট-আইকন (Forget, Input, Output) যুক্ত, প্রতিটি তথ্যপ্রবাহ নিয়ন্ত্রণ করছে
- নিচে ছোট ক্যাপশন: “Cell state acts as a highway with gated read/write/erase access”

LSTM Cell State (Information Highway)



১১.৩ Forget Gate

Forget Gate সিদ্ধান্ত নেয় আগের Cell State-এর কতটুকু “ভুলে যাওয়া” হবে। এটি বর্তমান ইনপুট ও আগের hidden state দেখে ০ (সম্পূর্ণ ভুলে যাওয়া) থেকে ১ (সম্পূর্ণ রাখা) এর মধ্যে একটি মান আউটপুট দেয়:

$$f_t = \sigma(W_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f)$$

১১.৪ Input Gate

Input Gate সিদ্ধান্ত নেয় বর্তমান time-step-এর কতটুকু নতুন তথ্য Cell State-এ যোগ করা হবে:

$$i_t = \sigma(W_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i)$$

১১.৫ Candidate Cell State

একটি “প্রার্থী” (candidate) নতুন তথ্য $\tanh$ -এর মাধ্যমে তৈরি করা হয়, যা Input Gate দ্বারা নিয়ন্ত্রিতভাবে Cell State-এ যোগ হবে:

$$\mathbf{C}_t = \tanh(W_C[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_C)$$

১১.৬ Cell State Update — Additive Update

এবার আগের Cell State-কে Forget Gate দিয়ে স্কেল করে, এবং নতুন candidate-কে Input Gate দিয়ে স্কেল করে — দুটো যোগ করে নতুন Cell State তৈরি হয়:

$$\mathbf{C}_t = \mathbf{f}_t \odot \mathbf{C}_{t-1} + \mathbf{i}_t \odot \mathbf{C}_t$$

এখানে $\odot$ হলো element-wise multiplication (Hadamard product)। এই সমীকরণটিই LSTM-এর সবচেয়ে গুরুত্বপূর্ণ বৈশিষ্ট্য — এটি একটি **additive** (যোগভিত্তিক) আপডেট, $\tanh$ -এর মধ্য দিয়ে repeated multiplicative transformation নয়। এই পার্থক্যটিই সেকশন ১২-তে গ্রেডিয়েন্ট প্রবাহের উন্নতির মূল কারণ হিসেবে ব্যাখ্যা করা হয়েছে।

১১.৭ Output Gate

Output Gate সিদ্ধান্ত নেয় Cell State-এর কতটুকু অংশ বর্তমান hidden state / output হিসেবে প্রকাশ পাবে:

$$\mathbf{o}_t = \sigma(W_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o)$$

১১.৮ Hidden State

চূড়ান্ত hidden state পাওয়া যায় Output Gate এবং $\tanh(\text{Cell State})$ -এর element-wise গুণফল থেকে:

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{C}_t)$$

প্রতীক	অর্থ
$\mathbf{f}_t$	Forget Gate আউটপুট (০-১, কতটুকু পুরনো Cell State রাখা হবে)
$\mathbf{i}_t$	Input Gate আউটপুট (০-১, কতটুকু নতুন তথ্য যোগ হবে)
$\mathbf{C}_t$	Candidate Cell State (নতুন সম্ভাব্য তথ্য, $\tanh$ দিয়ে তৈরি)
$\mathbf{C}_t$	Cell State (দীর্ঘমেয়াদি মেমোরি)
$\mathbf{o}_t$	Output Gate আউটপুট (০-১, কতটুকু Cell State প্রকাশ পাবে)
$\mathbf{h}_t$	Hidden State (স্বল্পমেয়াদি আউটপুট)
σ	Sigmoid activation — গেটের জন্য ব্যবহৃত
$\odot$	Element-wise (Hadamard) multiplication

Sigmoid বনাম tanh — কেন দুটো ভিন্ন Activation?

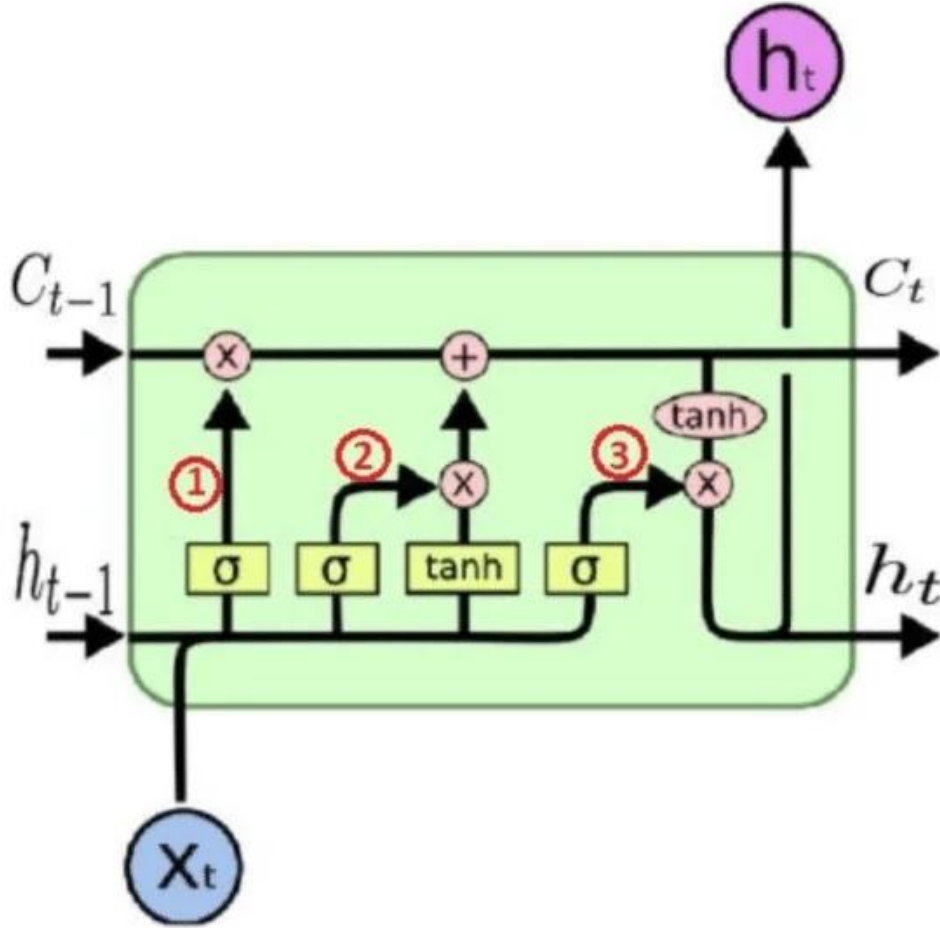
Function	Output Range	মূল ভূমিকা (Main Role)
Sigmoid (σ)	0 থেকে 1	Gate নিয়ন্ত্রণ — “কতটুকু দিয়ে যেতে দেওয়া হবে” তা প্রকাশ করে
tanh	-1 থেকে 1	Candidate / hidden representation — ধনাত্মক ও ঋণাত্মক উভয় তথ্য ধারণ করতে পারে

Sigmoid গেটের জন্য উপযুক্ত কারণ এর আউটপুট সরাসরি “কত শতাংশ (fraction) দিয়ে যেতে দেওয়া হবে” হিসেবে ব্যাখ্যা করা যায় (0 = সম্পূর্ণ বন্ধ, 1 = সম্পূর্ণ খোলা)। অন্যদিকে tanh candidate/hidden representation-এর জন্য উপযুক্ত কারণ এটি চিহ্নযুক্ত (signed) তথ্য বহন করতে পারে — শুধু “কতটুকু” নয়, বরং “কোন দিকে” তথ্যও প্রকাশ করে।

চিত্র / Figure 14: সম্পূর্ণ LSTM Cell

এই ডায়াগ্রামে যা দেখানো হবে (What this diagram should show):

- একটি একক বড় বাক্সে LSTM cell-এর সম্পূর্ণ ভেতরের গঠন আঁকা
- ইনপুট: x_t , h_{t-1} , C_{t-1} — বামদিক থেকে প্রবেশ করছে
- তিনটি সিগময়েড গেট (Forget, Input, Output) এবং একটি tanh candidate ব্লক পরিষ্কারভাবে লেবেলযুক্ত
- Cell State আপডেট ($\odot$ ও $+$ অপারেশন সহ) কেন্দ্রীয় অনুভূমিক রেখা বরাবর দেখানো
- আউটপুট: C_t , h_t — ডানদিকে বের হচ্ছে
- এটি হলো এই নোটের সবচেয়ে গুরুত্বপূর্ণ ডায়াগ্রাম — standard/conventional LSTM cell diagram অনুসরণ করা আবশ্যিক



Common Mistakes

- Hidden State ($\mathbf{h}_t$) এবং Cell State ($\mathbf{C}_t$)-কে একই জিনিস ভেবে গুলিয়ে ফেলা — $\mathbf{C}_t$ দীর্ঘমেয়াদি মেমোরি, $\mathbf{h}_t$ তার একটি gated, সংক্ষিপ্ত “projection” মাত্র।
- তিনটি গেটের ফাংশন গুলিয়ে ফেলা — মনে রাখার সহজ উপায়: Forget = পুরনো তথ্য মুছেবে কি না, Input = নতুন তথ্য যোগ হবে কি না, Output = বর্তমানে কতটুকু প্রকাশ পাবে।
- Sigmoid-এর আউটপুটকে সবসময় “প্রকৃত সম্ভাবনা (probability)” হিসেবে ব্যাখ্যা করা — গেটের প্রেক্ষাপটে এটি একটি নিয়ন্ত্রণ-মান (control signal), classification probability নয়।

১২. কেন LSTM সাহায্য করে (Why LSTM Helps)

১২.১ Additive Update-এর গুরুত্ব

Vanilla RNN-এ hidden state প্রতিটি time-step-এ একটি সম্পূর্ণ নতুন tanh transformation-এর মধ্য দিয়ে যায় (multiplicative-এর মতো আচরণ করে) — যার ফলে গ্রেডিয়েন্ট বারবার W_h ও $\tanh'(z)$ দ্বারা গুণিত হয়। কিন্তু LSTM-এর Cell State আপডেট সমীকরণটি লক্ষ করুন:

$$\mathbf{C}_t = \mathbf{f}_t \odot \mathbf{C}_{t-1} + \mathbf{i}_t \odot \mathbf{C}_t$$

এখানে $\mathbf{C}_{t-1}$ থেকে $\mathbf{C}_t$ -তে যাওয়ার পথে কোনো ম্যাট্রিক্স গুণন বা tanh-এর মধ্য দিয়ে repeated transformation নেই — শুধু একটি element-wise scaling ($\mathbf{f}_t$ দিয়ে) এবং একটি addition। এটিই “additive update”।

১২.২ গ্রেডিয়েন্ট প্রবাহ (Gradient Flow)

Cell State-এর মধ্য দিয়ে গ্রেডিয়েন্টের local derivative আনুমানিকভাবে:

$$\frac{\partial \mathbf{C}_t}{\partial \mathbf{C}_{t-1}} \approx \mathbf{f}_t$$

গুরুত্বপূর্ণ পর্যবেক্ষণ

- যখন Forget Gate-এর মান 1-এর কাছাকাছি থাকে, তখন Cell State-এর মধ্য দিয়ে গ্রেডিয়েন্ট প্রায় অপরিবর্তিতভাবে (নিকটবর্তী গুণনীয়ক ≈ 1) পেছনে প্রবাহিত হতে পারে — Vanilla RNN-এর মতো প্রতি ধাপে $W_h \cdot \tanh'(z)$ -এর পুনরাবৃত্ত গুণনের কারণে সৃষ্ট দ্রুত সংকোচন এখানে হয় না। এই তুলনামূলক “সরাসরি” (direct) পথটিই LSTM-কে দীর্ঘ সিকোয়েন্সে ভালোভাবে শিখতে সক্ষম করে।

সতর্কতা: LSTM vanishing gradient সমস্যা সম্পূর্ণভাবে দূর করে না — এটি শুধুমাত্র vanishing-gradient সমস্যাকে উল্লেখযোগ্যভাবে প্রশমিত (mitigate) করে এবং দীর্ঘমেয়াদি নির্ভরতা শেখাকে ব্যবহারিকভাবে অনেক বেশি সম্ভব করে তোলে। Forget Gate-এর মান যদি 1-এর থেকে যথেষ্ট দূরে থাকে, তাহলে LSTM-এও গ্রেডিয়েন্ট সংকুচিত হতে পারে।

১২.৩ দীর্ঘমেয়াদি নির্ভরতা (Long-Term Dependency)

সহজ উদাহরণ: “The movie was not good because the acting was poor.”

এই বাক্যে “not” শব্দটি বাক্যের শুরুর দিকে থাকলেও, পরবর্তী “good” শব্দের অর্থ সম্পূর্ণভাবে পরিবর্তন করে দেয়। একটি ভালো sequence model-কে “not” মনে রাখতে হবে যতক্ষণ না সে “good” শব্দে পৌঁছায় — এটিই long-term dependency। ধারণাগতভাবে, LSTM-এর Cell State “not”-এর প্রভাব সংরক্ষণ করে রাখতে পারে (Forget Gate একে মুছে না ফেললে), যতক্ষণ না প্রাসঙ্গিক পরবর্তী শব্দ আসে।

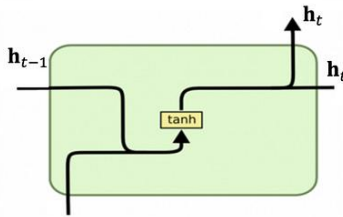
লক্ষণীয়: এই একটিমাত্র উদাহরণ কোনোভাবেই গাণিতিকভাবে প্রমাণ করে না যে LSTM Vanilla RNN-এর চেয়ে “শ্রেষ্ঠ” — এটি শুধু ধারণাটি স্বজ্ঞামূলকভাবে বোঝানোর জন্য একটি উদাহরণ মাত্র।

চিত্র / Figure 15: Vanilla RNN বনাম LSTM — ধারণাগত তুলনা

এই ডায়াগ্রামে যা দেখানো হবে (What this diagram should show):

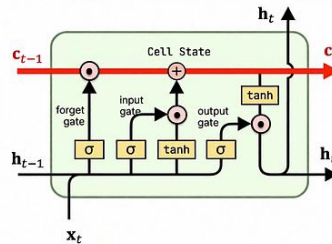
- বামে: Vanilla RNN, $h_{t-1} \rightarrow h_t$ একটি single tanh transformation-এর মধ্য দিয়ে, প্রতি ধাপে সম্পূর্ণ রূপান্তরিত
- ডানে: LSTM, $C_{t-1} \rightarrow C_t$ একটি অপেক্ষাকৃত “সরাসরি”, gated পথের মধ্য দিয়ে (হাইওয়ে-সদৃশ)
- দুই পাশে ছোট ক্যাপশন: “Repeated multiplicative transformation” বনাম “Gated additive pathway”

Vanilla RNN



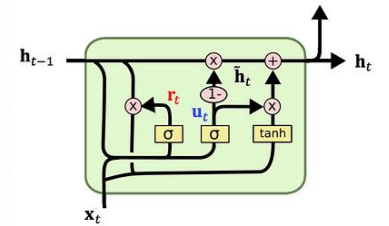
- **Pros:** Fast, Simple.
- **Cons:** Vanishes Gradients.
- **Best For:** Short sequences.

LSTM



- **Pros:** Powerful, Long memory.
- **Cons:** Slow, Heavy compute.
- **Best For:** Complex tasks (Translation).

GRU



- **Pros:** Efficient, Fast convergen
- **Cons:** Less expressive?
- **Best For:** Mobile / Real-time.

Key Takeaways

- LSTM-এর Cell State আপডেট additive ($C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$), multiplicative transformation-এর repeated চেইন নয়।
- $\partial C_t / \partial C_{t-1} \approx f_t$ — Forget Gate 1-এর কাছাকাছি থাকলে গ্রেডিয়েন্ট তুলনামূলক সরাসরি প্রবাহিত হতে পারে।
- LSTM vanishing gradient সমস্যা সম্পূর্ণ দূর করে না, বরং উল্লেখযোগ্যভাবে প্রশমিত করে।
- দীর্ঘমেয়াদি নির্ভরতা (long-term dependency) শেখার ক্ষেত্রে LSTM Vanilla RNN-এর তুলনায় ব্যবহারিকভাবে অনেক বেশি সক্ষম।

১৩. RNN বনাম LSTM — সারসংক্ষেপ তুলনা

নিচের তুলনাটি একটি সুষম (balanced) দৃষ্টিভঙ্গি উপস্থাপন করে — উদ্দেশ্য “RNN খারাপ, LSTM ভালো” এই অতি-সরলীকৃত সিদ্ধান্তে পৌঁছানো নয়, বরং প্রতিটি আর্কিটেকচারের বাস্তব trade-off বোঝা।

বৈশিষ্ট্য (Feature)	Vanilla RNN	LSTM
মেমোরি (Memory)	শুধু Hidden State	Hidden State + Cell State
গেট (Gates)	নেই	Forget + Input + Output
আপডেট পদ্ধতি	Recurrent transformation (tanh-ভিত্তিক)	Gated additive update
দীর্ঘমেয়াদি নির্ভরতা	কঠিন	তুলনামূলক ভালোভাবে সমর্থিত
Vanishing Gradient	অধিক তীব্র	উল্লেখযোগ্যভাবে প্রশমিত (সম্পূর্ণ দূর নয়)
প্যারামিটার সংখ্যা	কম	বেশি (প্রায় ৪ গুণ, তিনটি গেট + candidate-এর কারণে)
গাণিতিক ও কম্পিউটেশনাল জটিলতা	কম	বেশি
প্রশিক্ষণ গতি (একক step)	দ্রুততর	তুলনামূলক ধীর (বেশি প্যারামিটার)

Observation

- LSTM সবসময় Vanilla RNN-এর চেয়ে “সেরা পছন্দ” নয় — ছোট, স্বল্প-নির্ভরতার সিকোয়েন্সে Vanilla RNN দ্রুততর ও সরলতর সমাধান হতে পারে। LSTM-এর মূল সুবিধা প্রকাশ পায় যখন সিকোয়েন্স দীর্ঘ এবং long-term dependency গুরুত্বপূর্ণ।

১৪. Common Mistakes — সম্পূর্ণ তালিকা

এই নোট জুড়ে প্রতিটি প্রধান সেকশনের পরে প্রাসঙ্গিক Common Mistakes বক্স দেওয়া হয়েছে। নিচে পরীক্ষার আগে দ্রুত রিভিশনের জন্য সবগুলো একত্রে দেওয়া হলো:

Sequential Data ও ANN

- মনে করা যে ANN বড় বা বেশি লেয়ার ব্যবহার করলেই sequence বুঝতে পারবে।
- Bag-of-Words-কে sequence-aware representation ভেবে ভুল করা।

Data প্রস্তুতকরণ

- প্যাডিং ভ্যালু (0)-কে প্রকৃত ইনপুট ডাটা হিসেবে গণনা করা — masking ছাড়া ভুল ফলাফল হতে পারে।
- One-hot vector-কে বড় vocabulary-তেও “সস্তা ও যথেষ্ট” ভাবা।
- Word Embedding-কে semantic understanding-এর নিখুঁত সমাধান ভাবা।

RNN আর্কিটেকচার ও Forward Pass

- W_x এবং W_h -এর dimension এক ভেবে ভুল করা — W_h সবসময় square matrix ($n_h \times n_h$)।
- প্রতিটি time-step-এ নতুন ওয়েট ম্যাট্রিক্স তৈরি হয় ভাবা — বাস্তবে parameter sharing হয়।

BPTT ও গ্রেডিয়েন্ট

- Gradient-কে Updated Parameter-এর সাথে গুলিয়ে ফেলা।
- BCE সমীকরণের ঋণাত্মক চিহ্ন ভুলে যাওয়া।
- Shared parameter-এর গ্রেডিয়েন্ট আলাদা আলাদা রাখা, accumulate (যোগ) না করা।
- $\tanh'(z)$ সবসময় 0–0.25-এর মধ্যে থাকে ভাবা — এই সীমা sigmoid-এর জন্য প্রযোজ্য, $\tanh'(0) = 1$ ।
- Gradient Clipping-কে Vanishing Gradient-এরও সমাধান ভাবা — এটি শুধুমাত্র Exploding Gradient-এর জন্য কার্যকর।

LSTM

- Hidden State ও Cell State-কে একই জিনিস ভাবা।
- তিনটি গেটের ভূমিকা গুলিয়ে ফেলা (Forget vs Input vs Output)।
- Sigmoid-এর আউটপুটকে সবসময় “সম্ভাবনা (probability)” হিসেবে ব্যাখ্যা করা।
- মনে করা যে LSTM vanishing gradient সম্পূর্ণভাবে দূর করে দেয় — এটি শুধু উল্লেখযোগ্যভাবে প্রশমিত করে।

- RNN বনাম LSTM তুলনাকে “RNN খারাপ, LSTM ভালো” এই অতি-সরলীকরণে নামিয়ে আনা।

১৫. Final Revision Sheet

পরীক্ষার ঠিক আগে দ্রুত রিভিশনের জন্য এই সেকশনটি ডিজাইন করা হয়েছে — সম্পূর্ণ নোট আবার না পড়েই মূল ধারণা, সমীকরণ, এবং সংজ্ঞাগুলো এক নজরে দেখে নেওয়া যাবে।

১৫.১ Essential Concepts

- Sequential Data — যেখানে ক্রম/অবস্থান অর্থ বহন করে।
- ANN-এর সীমাবদ্ধতা — মেমোরি নেই, ক্রম ধারণ করতে অক্ষম, fixed-length input।
- RNN = ANN + Memory (Hidden State + Recurrence + Parameter Sharing)।
- Zero Padding ও Masking — variable-length সিকোয়েন্স ব্যাচে প্রসেস করার কৌশল।
- One-Hot vs Word Embedding — sparse/orthogonal বনাম dense/semantic।
- চারটি RNN আর্কিটেকচার: One-to-One, One-to-Many, Many-to-One, Many-to-Many।
- Forward Propagation: pre-activation $\rightarrow \tanh \rightarrow$ hidden state $\rightarrow$ output।
- BPTT — unroll করে backpropagation, shared parameter-এর গ্রেডিয়েন্ট accumulate হয়।
- Vanishing/Exploding Gradient — repeated multiplication-এর ফল; Gradient Clipping শুধু exploding-এর সমাধান।
- LSTM — Cell State + তিনটি Gate (Forget, Input, Output) + Additive Update।

১৫.২ Essential Equations

Topic	Formula
RNN Pre-activation	$z_{h,t} = x_t W_x + h_{t-1} W_h + b_h$
RNN Hidden State	$h_t = \tanh(z_{h,t})$
RNN Output	$\hat{y}_t = \sigma(h_t W_y + b_y)$
Binary Cross-Entropy	$L = -[y \ln(\hat{y}) + (1 - y) \ln(1 - \hat{y})]$
Sigmoid + BCE Gradient	$\frac{\partial L}{\partial z_o} = \hat{y} - y$
Shared Parameter Gradient	$\frac{\partial L}{\partial W_h} = \sum_t \frac{\partial L_t}{\partial W_h}$

Gradient Descent Update	$\theta_{\text{new}} = \theta_{\text{old}} - \eta \frac{\partial L}{\partial \theta}$
Gradient Clipping	if $\ g\ > \tau$: $g \leftarrow g \frac{\tau}{\ g\ }$
LSTM Forget Gate	$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$
LSTM Input Gate	$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$
LSTM Candidate	$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C)$
LSTM Cell Update	$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$
LSTM Output Gate	$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$
LSTM Hidden State	

১৫.৩ Important Definitions

টার্ম	সংজ্ঞা
Hidden State	RNN-এর স্বল্পমেয়াদি মেমোরি, প্রতি time-step-এ আপডেট হয়
Cell State	LSTM-এর দীর্ঘমেয়াদি মেমোরি, gated additive পথে প্রবাহিত হয়
Parameter Sharing	একই ওয়েট ম্যাট্রিক্স সব time-step-এ পুনরায় ব্যবহার
BPTT	RNN unroll করে backpropagation প্রয়োগ করা
Vanishing Gradient	বারবার ছোট factor-এর গুণনে গ্রেডিয়েন্ট শূন্যের কাছে চলে যাওয়া
Gradient Clipping	গ্রেডিয়েন্ট norm একটি সীমার মধ্যে সীমিত রাখা, দিক অপরিবর্তিত রেখে

১৫.৪ Important Differences

তুলনা	মূল পার্থক্য
One-Hot vs Embedding	Sparse/orthogonal বনাম Dense/semantic-aware
Local vs Temporal Gradient	একক time-step-এর গ্রেডিয়েন্ট বনাম সময়ের সাপেক্ষে প্রবাহিত গ্রেডিয়েন্ট
Vanishing vs Exploding Gradient	গ্রেডিয়েন্ট সংকুচিত হওয়া বনাম exponentially বৃদ্ধি পাওয়া
Hidden State vs Cell State	স্বল্পমেয়াদি, gated output বনাম দীর্ঘমেয়াদি, তুলনামূলক সরাসরি মেমোরি-পথ
Vanilla RNN vs LSTM Update	Multiplicative (tanh transformation) বনাম Additive (gated sum)